

An Ensemble Approach for Anomaly Detection in Hand-Mounted PCB Components

Master Thesis

MSc Business Administration and Data Science

Course Code: CDSCO4001E

Frederik Markus Rytøft Gøtske (frgo18ab - S128816)

Aleksander August Fowler Torp (alto22ad - S158277)

Copenhagen Business School

May 15, 2024

85 pages, 162 465 characters with spaces

Supervisor: Nicolai Frost Jacobsen

Abstract

In collaboration with a small-scale PCB manufacturer, this thesis explores the design and implementation of a cost-efficient hardware and software setup for anomaly detection in hand-mounted PCB components. Recognizing the unique needs of such manufacturers, this research develops a customized automated optical inspection system that incorporates multiple one-class Convolutional Autoencoder models, each specialized for detecting anomalies in a specific type of PCB components, without requiring anomaly images for training. These are supported by a YOLOv8-based cropping method that accurately identifies and isolates components of interest. The system achieves an F1-Score of 0.99 with a training set of 200 images, illustrating the system's potential adaptability and accuracy across various PCB designs. The full research process is documented, extensively covering the data collection, model training, evaluation, and deployment phases. The study also features the insights learned from the transition from a simplistic camera setup, susceptible to frequent errors and inconsistencies, to a stable and consistent imaging system. The improved imaging capability, combined with the tailored approach of using specialized models provides a flexible, scalable and economically viable solution for enhancing quality control in PCB manufacturing, addressing both the cost concerns and the technical requirements of the manufacturer. Finally, the proposed system is split into a training and operational pipeline for easy deployment and further development.

Acknowledgements

We would like to express our gratitude to those who enabled the finalization of this project and who has been essential throughout this thesis by lending us their guidance and expertise. That is, our supervisor, Nicolai Frost Jacobsen, the partner company [REDACTED], herein especially [REDACTED] and [REDACTED] as well as the Danish e-infrastructure Consortium and CBS Research Data Management Support team for providing us with the necessary cloud computing software under Project *DeiC-CBS-L1-000060* and continuous technical assistance to resolve technical challenges along the way.

Contents

1	Introduction	1
1.1	Partner Company	1
1.2	Problem Formulation	2
1.3	Thesis Outline	3
1.4	Disclaimers	3
2	Conceptual Framework	3
2.1	Artificial Neural Networks	3
2.1.1	Learning in Neural Networks	5
2.1.2	Activation Function	7
2.1.3	Loss Function	8
2.1.4	Weight Optimization	10
2.1.5	Supervised vs. Unsupervised Learning	12
2.1.6	Transfer Learning and Fine-tuning	13
2.2	Convolutional Neural Networks	14
2.2.1	YOLOv8	17
2.2.2	VGGNet	18
2.3	Autoencoders	19
2.3.1	Convolutional Autoencoders	19
2.4	Imaging Tasks	20
2.4.1	Image Similarity	20
2.4.2	Image Registration	21
2.4.3	Image Classification	22
3	Literature Review	25
3.1	Anomaly Detection	25
3.1.1	Inherent Challenges	25
3.1.2	Transition to Deep Learning	26
3.2	Deep Learning Approaches for Anomaly Detection	27
3.2.1	Overarching Strategies	27

3.2.2	One-Class Models	29
3.2.3	Large Multimodal Models	32
3.2.4	Supervised YOLO-based Models	33
3.2.5	Selected Approaches	33
4	Methodology	34
4.1	Research Framework	34
4.1.1	Research Philosophy	34
4.1.2	Research Approach	35
4.1.3	Methodological Research Choice	35
4.1.4	Research Strategy	36
4.1.5	Research Time Horizon	36
4.1.6	Techniques and Procedures	36
4.2	CRISP-DM Framework	37
4.3	Business Understanding	37
4.4	Data Understanding	38
4.4.1	Data Collection	38
4.4.2	Data Description	44
4.5	Data Preparation	45
4.5.1	Data Formatting and Initial Preprocessing	45
4.5.2	Partitioning for Grid Models	47
4.6	Data Modelling	48
4.6.1	CAE Architecture	49
4.6.2	CNN Architecture	52
4.6.3	Training Setup	54
4.7	Evaluation of Models	54
4.7.1	Grid Models	55
4.7.2	Component-Instance Models	56
5	Results	57
5.1	Model Performance Overview	57

5.2	Grid-Wise Models	58
5.3	Template-Cropping Models	59
5.3.1	TC-CNN Model	59
5.3.2	TC-CAE Model	59
5.4	YOLO-Cropping Models	60
5.4.1	YOLO-CNN Model	60
5.4.2	YOLO-CAE Model	61
6	Deployment and Application	61
6.1	Model Preparation and Training	62
6.1.1	YOLO Model Fine-tuning	62
6.1.2	Cropping and Saving Component-Instance Images	63
6.1.3	Component-Instance Model Training	63
6.1.4	Model Training Evaluation	64
6.2	Operational Deployment and Results Analysis	65
6.2.1	Component-Instance Identification and Model Application	66
6.2.2	Evaluating the Results	67
6.2.3	Anomaly Visualization	67
7	Discussion	68
7.1	Deep Dive of Final Outputs	68
7.2	Understanding Detection Errors and Training Data Quality	69
7.2.1	Anomaly Detection Causes	69
7.2.2	The Effects of Imperfect Training Data	71
7.3	Implications of Utilizing YOLO	73
7.3.1	Advantages of Integrating YOLO with CAEs	73
7.3.2	Comparative Performance Analysis	73
7.4	Implications of Using Complex Approaches	74
7.4.1	Using VAEs for AD	74
7.4.2	Using LMMs for AD	75
7.5	Implications of Grid-Wise and Perceptual Methods	76

7.5.1	Main Model	76
7.5.2	Baseline Model	77
7.5.3	Comparison	78
8	Limitations	79
9	Conclusion	81
A	Appendix	86

List of Acronyms

AAE Adversarial Autoencoder

ABC The Partner Company

AD Anomaly Detection

AOI Automated Optical Inspection

BN Batch Normalization

CAE Convolutional Autoencoder

CNN Convolutional Neural Network

CRISP-DM Cross Industry Standard Process for Data Mining

GAN Generative Adversarial Network

GM-CAE Grid-wise MSE-loss Convolutional Autoencoder

GP-CAE Grid-wise Pixel-loss Convolutional Autoencoder

IoU Intersection Over Union

LMM Large Multimodal Model

MSE Mean Squared Error

NN Artificial Neural Network

PCB Printed Circuit Board

POC Proof-of-Concept

RANSAC Random Sample Consensus

ReLU Rectified Linear Unit

SGD Stochastic Gradient Descent

SIFT Scale-Invariant Feature Transform

SSIM Structural Similarity Index Measure

TC-CAE Template-based Component-wise Convolutional Autoencoder

TC-CNN Template-based Component-wise Convolutional Neural Network

VAE Variational Autoencoder

YOLO-CAE YOLO-based Component-wise Convolutional Autoencoder

YOLO-CNN YOLO-based Component-wise Convolutional Neural
Network

1 Introduction

The drive towards automation in printed circuit board (PCB) manufacturing has accelerated the need for advanced solutions capable of detecting anomalies that human inspectors might miss, and for which current industrial automated optical inspection (AOI) systems might not cover. The manufacturing of PCBs is a complex process involving the attachment of various electronic components. Traditionally, quality control has depended heavily on human inspection and automated systems using carefully crafted manual templates, which are prone to error (Mirapeix et al., 2007). For small-scale manufacturers, investing in advanced AOI technologies can pose significant financial challenges. However, the cost of not addressing these potential defects can lead to increased waste and safety risks.

This thesis investigates two different modeling approaches: component-wise, targeting specific PCB components, and grid-wise, dividing the PCB into uniform grids. Additionally, it explores model architectures based on both Convolutional Neural Networks and Convolutional Autoencoders. This exploration leads to the introduction of a working proof-of-concept (POC) that employs an ensemble approach to detect anomalies in hand-mounted components on PCBs using the component-wise method with a pretrained computer vision model, YOLO, to accurately crop out components, and several specialized Convolutional Autoencoders, each fine-tuned to a specific type of component. Our work demonstrates a cost-efficient and scalable solution for small-scale PCB manufacturers, adaptable to a variety of PCB products with minimal configuration requirements. Structured around the CRISP-DM framework and the research onion model, the project describes all stages from data collection to model evaluation and deployment.

1.1 Partner Company

The partner company ABC (ABC) is a Norwegian company that specializes in the production of PCBs, [REDACTED]

[REDACTED] [REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

1.2 Problem Formulation

As a measure to improve quality control over a broad span of product lines, ABC is interested in exploring cost-efficient applications of computer vision in its production process and viability in aiding quality inspection of PCBs.

ABC is looking to lay the initial foundation for exploring what alternative hardware and software solutions can be utilized for anomaly detection (AD) in hand-mounted PCB components. Within this context, this paper addresses the following research question:

How can a cost-efficient hardware and software setup be designed and implemented for anomaly detection in hand-mounted PCB components for small-manufacturers, considering the specific needs and constraints given by ABC?

Topic Delimitation This thesis confines its exploration to the development of a computer vision system for AD in PCBs, specifically targeting hand-mounted components. The study is conducted using datasets created in collaboration with ABC. Insights from problems with the initial dataset’s quality and the subsequent enhancements form a significant part of the learning curve. The AD system designed in this thesis is intended to serve as a POC that can be scaled and adapted for broader application within the company. By demonstrating the feasibility and effectiveness of our approach in a controlled environment, we aim to lay the groundwork for future adaptations that can span different types of PCB designs and use cases.

1.3 Thesis Outline

The thesis is structured as follows: First, the conceptual framework presents relevant concepts essential for this project. Next, the literature review provides an overview of relevant research on AD and associated methods. Then, the methodology section elaborates extensively on the methods applied for replicability and reliability purposes. Following this, the results of the explored models are presented and compared. Subsequently, after identifying the most promising model, the development of the next stage of the POC, including systems and pipelines, is explained. Finally, important insights, limitations, and potential avenues for further exploration and refinement are discussed.

1.4 Disclaimers

1. This thesis is the product of a collaborative effort with our partner company, focusing specifically on integrating the company’s interests into an academic context. However, no form of compensation is provided to avoid any conflict of interest.
2. Throughout this paper, ChatGPT has been used as a sparring partner, both in terms of creating drafts and refining text as well as for practical help with implementing and troubleshooting code.

2 Conceptual Framework

This section presents the conceptual framework which underpins the concepts and methods applied in this project. We focus exclusively on those that are relevant to the experiments conducted and for the development of a POC as to provide ABC with an understanding of relevant mechanisms in an accessible manner.

2.1 Artificial Neural Networks

Artificial neural networks (NN) are inspired by the biological neural networks of animal brains. NNs are used to approximate unknown functions, that can depend on a large number of inputs, to describe how input is mapped to a given output. NNs are particularly useful in machine

learning for modeling complex patterns through layers of nodes (or neurons), interconnected to form a network architecture. Each neuron within an NN transforms input data into more useful outputs, which are then passed on to subsequent layers, effectively enabling the NN to independently learn a mapping function from observational data (Aggarwal, 2023). At their core, NNs are built up of the following components:

1. **Input Layer:** The initial layer consists of passive neurons that pass the raw input data to the subsequent layers without any modification. Each neuron represents one feature from the input dataset. For example, an image with 10×10 pixels, equal 100 features and the input layer must then have 100 neurons.
2. **Weights and Bias:** Each connection between neurons in successive layers is associated with a weight. These weights are adjusted as the network learns, changing how influential one neuron's output can be on another. Next, neurons commonly include a bias term, which allows the output to shift independently of the weighted input values. This bias term acts like an intercept in a linear equation, enabling the neuron to adjust its activation threshold. Allowing neurons to activate for patterns not centered around zero improves the NNs flexibility and performance in recognizing complex relationships.
3. **Activation Function:** An essential function which helps determine which neurons should be active, i.e., produce output values, within each layer of the network. It also introduces non-linearity, and without it, the network cannot learn and represent complex non-linear patterns and relationships.
4. **Hidden Layers:** The layers between the input and output layers are called hidden layers. These layers process data using weights, biases, and activation functions. The number of hidden layers equal the depth of a NN, which influences its ability to learn complex patterns.
5. **Output Layer:** The number of neurons in the final layer must equal the number of possible output categories. For instance, a single neuron is sufficient for performing regression, whereas binary prediction requires two neurons.
6. **Loss Function:** A loss function is used improve the NN by quantifying how wrong the NNs predictions are, which is the difference between the network's predicted values and the actual target values.

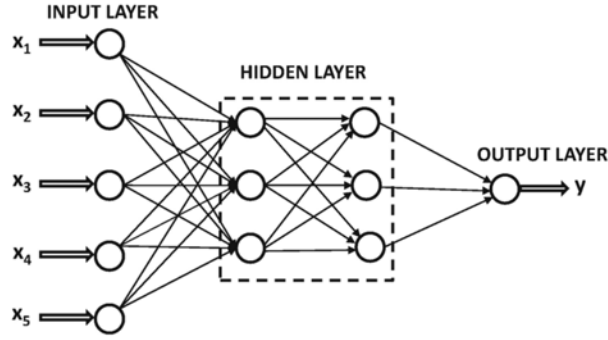


Figure 1: Basic NN illustration from Aggarwal (2023)

NNs can be applied to various types of data, including tabular data, images, and text, enabling tasks like image recognition, language translation, and predictive modelling with high efficiency and accuracy. The adaptability to different data types is primarily due to the flexible architecture and the capability of NNs to autonomously learn inherent patterns in data. The fundamentals of NNs, a feed-forward network with a single layer of neurons, can mathematically be expressed as:

$$y = f(\mathbf{W} \cdot \mathbf{X} + b) \quad (1)$$

where:

- f is the non-linear activation function.
- $\mathbf{W} = [w_1, w_2, \dots, w_n]$ is the weights vector, each weight w_i corresponds to a feature in the input vector and determines the influence it has on the output.
- $\mathbf{X} = [x_1, x_2, \dots, x_n]$ is the input vector, where each x_i is a feature of the input data.
- b is the bias, which shifts the activation function curve.

2.1.1 Learning in Neural Networks

Learning in neural networks involves a series of stages that collectively enable the network to understand patterns in data (that is, approximate an optimal mapping function). This process requires proper data preparation to ensure high-quality input, the efficient forward propagation of information through the network layers, and careful backward propagation to refine the model's internal parameters. During these steps, challenges like overfitting, and vanishing and exploding gradients may arise.

Data Preparation The preparation of data is foundational to the successful training of NNs. This stage encompasses a variety of tasks including the collection of data, cleaning to remove noise or irrelevant information, feature selection to identify the most important attributes that the model should focus on, and finally, data partitioning. Data partitioning involves splitting the dataset into three distinct sets: the training set, the validation set, and the testing set. The training set is used to fit the model. The validation set provides an unbiased evaluation of the model during the training phase and serves as a tool for model tuning, allowing for the adjustment of hyperparameters, in cases of over- or underfitting. These, respectively, occur when a model is either too closely aligned with training data or fails to capture relevant patterns, both scenarios hinder its ability to generalize effectively. Here, *hyperparameter* refers to the parameters that guide the training of a model, and *parameter* refers to the internal model parameters which are updated during training. Lastly, the test set is used after the model has been trained to obtain an unbiased assessment of the model’s performance on unseen data. This project’s data preparation is described in Section 4.5.

Forward Propagation Once the data is prepared, it is propagated forward through the NN. Each neuron computes an output based on its input data, weights, and bias. This output is then passed on to neurons in the subsequent layer, and the process continues until the final layer produces the network’s output (Géron, 2023). Along the way, each layer applies an activation function to introduce non-linearity (see Section 2.1.2), enabling the network to learn and model complex relationships within the data. The forward propagation phase ensures that all necessary computations for the model’s output are completed, laying the groundwork for subsequent error calculations in backpropagation (Géron, 2023).

Backward Propagation Backpropagation is an algorithm used to update the model parameters (Géron, 2023). It starts by comparing the network’s actual output with the expected output using a loss function (see Section 2.1.3), which quantifies the error between the two. Then, the contribution of each connection (weight) and bias in the final layer to the prediction error is calculated using the chain rule. The rule determines how changes in one layer impact the output by tracing the relationships between different layers. Next, it computes how errors in the subsequent layer were influenced by the weights and biases in the layer before it. By

repeating this step-by-step error measurement backward to the input layer, backpropagation identifies how each weight and bias needs to be adjusted to reduce the overall error in the network.

After computing the necessary adjustments for all weights and biases, a weight optimization step (such as gradient descent, see Section 2.1.4) applies these corrections, moving the network closer to an optimal solution. This process of making predictions (forward propagation), measuring the error, and then applying corrections (backpropagation) is repeated multiple times over mini-batches of data during training until the network's error is minimized and the model learns the relationships in the data (Géron, 2023).

Vanishing and Exploding Gradients Think of the gradient as the slope of the error curve from the weight optimization step. It tells you how steeply the error changes when you adjust the model's weights. A small gradient means the slope is gentle, and a large gradient means the slope is steep. In the vanishing gradients problem, the slope becomes so gentle that the model doesn't learn, while in the exploding gradients problem, the slope becomes so steep that the model's learning process becomes unstable (Géron, 2023). These problems were highlighted by Glorot and Bengio (2010), in which they identified poor weight initialization and saturating activation functions like the sigmoid as key contributors (see Section 2.1.2).

2.1.2 Activation Function

The activation function f is applied to the output of each neuron in the network, and it determines whether a given neuron should be activated or not, similar to a light switch. It also introduces non-linearity, and is essential for a NNs ability to understand and capture non-linear relationships in the data as well as for the backpropagation algorithm. Without some form of non-linearity, even a NN with a deep stack of layers become nothing more than many linear transformations chained together, and linear functions cannot solve complex problems (Géron, 2023). Commonly used activation functions include:

- **Sigmoid:** $f(x) = \frac{1}{1+e^{-x}}$, outputs values between 0 and 1.
- **Tanh** $f(x) = \frac{2}{1+e^{-2x}} - 1$, outputs values between -1 and 1.
- **ReLU** $f(x) = \max(0, x)$, outputs either zero or input value.

- **LeakyReLU:** $f(x) = x$ if $x > 0$, else αx , where $\alpha > 0$, outputs always a positive value.

As for convolutional neural networks (CNNs), further elaborated on in Section 2.2, Krizhevsky et al. (2012) show that ReLU-equipped networks achieve lower training error on image datasets compared to their counterparts using saturating functions like sigmoid and tanh, which have gradients that diminish near their limits, leading to the vanishing gradient problem. For example, the sigmoid function saturates at 0 and 1 for extreme input values, resulting in near-zero gradients and inhibiting backpropagation (Géron, 2023). However, the ReLU function can suffer from the problem known as *dying ReLU*, and as such, the Leaky ReLU activation function was introduced (Xu et al., 2015). Leaky ReLU allows a small, non-zero, constant gradient α (such as 0.01) when the unit is not active and x is less than 0, which helps to keep some of the gradients alive during the backpropagation process, preventing the neurons from dying.

Using Leaky ReLU can mitigate the vanishing gradient problem by maintaining a healthier gradient flow, ensuring that all neurons in the network remain active and learn from the back-propagated errors, potentially leading to better overall model performance (Géron, 2023). Both the ReLU and Leaky ReLU are used in the final models, as seen in Sections 4.6.1 and 4.6.2.

2.1.3 Loss Function

The output from the final layer of the NN is utilized to compute how it can be improved. A loss function quantifies the difference (distance) between the NN’s predicted outputs and the actual data labels, providing feedback on how much to adjust the NN’s weights through backpropagation. Different types of NN require distinct and tailored loss methods, and as such, the choice of loss function varies based on the nature of the problem at hand (Géron, 2023). Image-based tasks often use functions based on either pixel-wise differences or higher-level feature differences to measure the dissimilarity between the original and reconstructed images.

MSE The Mean Squared Error (MSE), computes the mean of the squares of pixel-wise differences between the input and reconstructed images. The MSE is particularly effective in emphasizing larger errors and is computationally efficient, making it a popular choice in various image-related applications.

The MSE, denoted as $L_2(x, \hat{x})$, is calculated as follows:

$$L_2(x, \hat{x}) = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

Here, x represents the original image, $\hat{x}$ denotes the reconstructed image, and N signifies the number of pixels in the image. MSE is a fundamental and simple method to quantify the performance of image reconstructions (Géron, 2023). MSE is used as the loss function for all component-wise models, and for half of the grid-wise models.

Perceptual Loss While the MSE offers a straightforward measure, it compares corresponding pixel values without accounting for spatial relationships between neighboring pixels. To address this, perceptual loss (content loss), introduced by Gatys et al. (2016), involves using higher-level features of images to determine the loss. Features can be e.g. edges, corners, and textures. The feature representations of an input image $\hat{y}$ and a reference image y are extracted from specific layers of a pre-trained NN (elaborated on in Section 2.2.2), where different layers extract different types of features. These representations, denoted as $\phi_j(\hat{y})$ and $\phi_j(y)$ for the j -th layer, capture complex patterns and textures not necessarily captured by MSE. The loss between the two images at a particular layer j is defined as:

$$l_{\phi,j}^{\text{feat}}(\hat{y}, y) = \frac{1}{C_j H_j W_j} \|\phi_j(\hat{y}) - \phi_j(y)\|_2^2$$

C_j , H_j , and W_j are the number of color channels, height, and width of the feature representations at layer j . This calculates the squared and normalized distance (loss) between the feature representations of the two images. The total perceptual loss is usually computed by summing the perceptual losses over multiple layers (different layers of a pre-trained NN extract different types of features from an image):

$$\mathcal{L}_{\text{Perceptual}} = \sum_{j \in \text{selected layers}} l_{\phi,j}^{\text{feat}}(\hat{y}, y)$$

Pixel and Perceptual Loss de Oliveira et al. (2023) successfully introduced a loss function that leverages both higher-level features and pixel differences, which encourages NNs specialized to reconstruct images to consider the content of an image as well. The loss function, denoted $\mathcal{L}_{\text{Total}}$, is the total information loss between two images as the sum of pixel- and perceptual-loss, where λ_1 and λ_2 are the respective the weighting of each. This allows the necessary emphasis of each component to be adjusted depending on the task. Also see Section 4.6.1 for further discussion of this type of loss function.

$$\mathcal{L}_{\text{Total}} = \lambda_1 \mathcal{L}_{\text{MSE}} + \lambda_2 \mathcal{L}_{\text{Perceptual}}$$

2.1.4 Weight Optimization

The error calculated by the loss function is used for backpropagation, in which the gradient of the loss with respect to each weight is computed. These gradients guide the subsequent adjustments of all the weights. An optimization algorithm iteratively update the weights, moving them in the direction that reduces the loss, and the size of the update at each step is determined by the hyperparameter *learning rate* (Géron, 2023).

Gradient Descent Gradient Descent is a basic optimization method, which can be conceptualized by imagining the loss as a hilly landscape where high points represent worse performance, and where the goal is to find the lowest valley. The algorithm calculates the slope (gradient) at the current position. If the slope points upward, you go in the opposite direction, effectively *walking downhill* (Géron, 2023). The size of each step is determined by the *learning rate*. If the steps are too large (a large learning rate), the algorithm may overshoot the lowest point and keep bouncing around without finding the best solution, and if the steps are too small (a small learning rate), it could take a long time to reach the bottom. The entire dataset is used to calculate the slope for each step, ensuring accurate, but time- and computationally expensive, updates. Faster alternatives include Stochastic Gradient Descent (SGD) and Adam optimization (Géron, 2023). See Figure 2 for an illustration.

Stochastic Gradient Descent SGD picks a small, random subset of training data (known as a mini-batch) to estimate the gradients instead of using the whole dataset each time. This accelerates training because updates are more frequent and are computationally cheaper (Géron, 2023). Sampling random subsets add randomness to the training process (making the *walk downhill* less predictable) which can help the model escape local minima (suboptimal solutions, think of being stuck in a *non-lowest valley*) and potentially lead to a better overall solution (Géron, 2023).

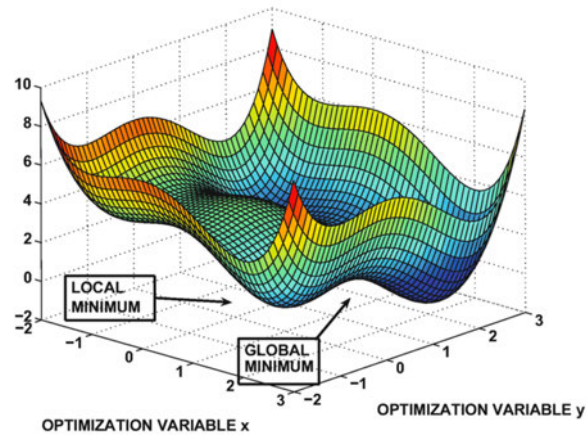


Figure 2: Gradient Descent (Aggarwal, 2023)

Adam Optimization Where SGD uses a fixed learning rate, Adam uses adaptive learning rates for each individual weight based on past gradients. It does so by tracking two key moving averages: the first moment (mean) and the second moment (variance) of the gradients. These moving averages help Adam adapt the learning rate dynamically by considering both the direction and magnitude of past gradient changes (Kingma & Ba, 2017). The adaptability of Adam can result in faster convergence (finding the *lowest valley*) and more stable training, particularly in models with complex architectures or highly variable data distributions. It strikes a balance between speed and accuracy by leveraging the strengths of SGD (frequent updates) and adaptive optimization (which is learning rate adjustments) (Géron, 2023). Adam optimization is used to train all models in this project, as seen in Section 4.6.3.

Batch Normalization Batch normalization (BN) was introduced by Ioffe and Szegedy (2015) to improve the training process of NNs. BN works by normalizing the inputs to a layer for each mini-batch of data, as it calculates the mean and standard deviation of the inputs and

uses these values to normalize the data. After normalization, the inputs are scaled and shifted using two parameters learned by the network: one for scaling and one for shifting. This process allows the network to optimize the inputs for better learning.

Think of BN as a way to keep the training process smooth and steady, as unregulated data that flows through a network can become unbalanced and cause the training to slow down or become unstable. BN resets this balance by normalizing the inputs, making sure the data stays on a consistent scale. This consistency helps the network learn faster and more effectively, making the aforementioned hill in the optimization step smoother and easier to navigate. As such, BN allows the network to use higher learning rates, reduces the need for careful initialization of parameters and acts as a regularizer. Although BN introduces additional computational overhead due to the extra calculations, the benefits often outweigh this cost by significantly improving training performance and stability (Géron, 2023). BN is applied to the Convolutional Autoencoder-based models discussed in Section 4.6.1.

2.1.5 Supervised vs. Unsupervised Learning

NNs are broadly categorized based on the type of learning they employ, supervised or unsupervised, depending on the task and available data.

Supervised Learning In supervised learning, the model is trained with labeled data with the goal of predicting these labels as accurately as possible (Géron, 2023). For instance, email spam filtering and image classification commonly use supervised approaches, where labels help guide a model in learning (Géron, 2023). In this paper, the supervised YOLOv8 architecture, described in Section 2.2.1, is used for the component-wise approach to accurately crop out the components of interest, as seen in Sections 4.6.3 and 6.2.

Unsupervised Learning On the other hand, unsupervised learning involves training models with unlabeled data, and the model must discover the inherent structures in the data without guidance. Unsupervised learning is useful for discovering hidden patterns, clustering, dimensionality reduction, and feature learning. However, a challenge to effective learning is defining a suitable objective function that can drive the learning process. This often involves using al-

gorithms that can capture the distribution of the input data or the relationships between data points. Both the component-wise and grid-wise approaches use unsupervised models to simplify both training and adaptation to new PCB designs. One-class models, like the Convolutional Neural Network and Convolutional Autoencoder of this paper are unsupervised.

Semi-Supervised Learning Semi-supervised learning sits between supervised and unsupervised learning by using both labeled and unlabeled data. This approach is particularly advantageous when obtaining labeled data is costly or time-consuming but unlabeled data is plentiful. In a typical semi-supervised learning setup, the model uses the labeled data to establish an initial understanding of the task, and then incorporates the unlabeled data to refine or expand its generalization. The unlabeled data can help improve the robustness of the learned representations and can be used to regularize the model, reducing overfitting (De Albuquerque Filho et al., 2022).

2.1.6 Transfer Learning and Fine-tuning

Transfer learning involves reusing the weights of a pre-trained NN, except for the output layer which is replaced to fit a new task. For example, if you want to identify a new type of boat in a set of images, you can adjust a NN that has been trained to identify common types of boats. In other words, transfer learning uses the feature extraction capabilities learned from the original dataset and applies them to a separate task (Géron, 2023).

Fine-tuning involves initializing the weights of the entire network or selected layers with values derived from the pre-trained model, rather than starting from a random initialization. This is achieved by first *freezing* the early layers of the network, which are responsible for capturing generic features such as edges and textures that are useful across different tasks (Iman et al., 2023). The later layers, more attuned to specific features of the training dataset, are then either *unfrozen* or adjusted more significantly during training. These techniques are elaborated on in Sections 2.2.1 and 2.2.2, and used to train a component-cropping model in Section 6.1.1.

2.2 Convolutional Neural Networks

CNNs, introduced by LeCun et al. (1989), are a NNs specialized for processing data with a grid-like structure, such as images (consider images as 2D grids of pixels), illustrated in Figure 3. The convolutional layers, which form the core of CNNs, are designed to leverage this structure using local receptive fields, shared weights, and pooling layers (Géron, 2023).

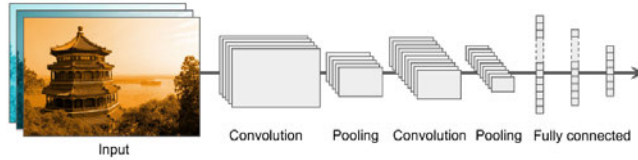


Figure 3: High-level overview of a common CNN architecture (Géron, 2023)

Convolutional Layers Convolutional layers are the core components of CNNs that utilize filters (or kernels) applied to input images through a sliding window mechanism. Each filter operates over a local receptive field, illustrated in Figure 4, computing a weighted sum to detect specific patterns like edges, textures, or shapes. The stride, which determines the filter’s step size, directly affects the spatial resolution of the resulting feature map, influencing the granularity of the detected patterns (Géron, 2023).

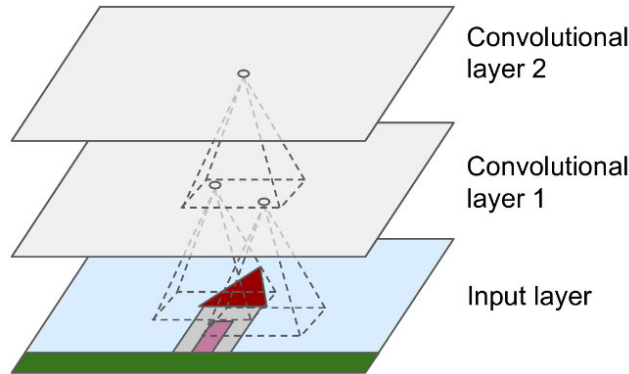


Figure 4: CNN: Layers and local receptive fields (Géron, 2023)

The convolutional operation allows each layer to learn increasingly complex and abstract features by stacking multiple layers. The initial layers typically identify low-level patterns, such as edges and corners, while deeper layers recognize higher-level patterns like object parts and complex shapes. After passing through several convolutional layers, feature maps are further processed by pooling layers, which reduce dimensionality while retaining essential information.

This hierarchical approach ensures the network efficiently identifies intricate structures and distinctive patterns within the data (Géron, 2023).

Pooling Layers After one or more convolutional layers, pooling layers reduce the spatial dimensions of feature maps. This helps decrease the number of parameters and computational requirements, which, in turn, helps prevent overfitting. The most common type is max pooling, where the maximum value is chosen from each region covered by the filter (Géron, 2023). For example, a max pooling layer with a 2×2 filter and a stride of 2 halves the dimensions in each direction, leading to a fourfold reduction in area, as illustrated in Figure 5.

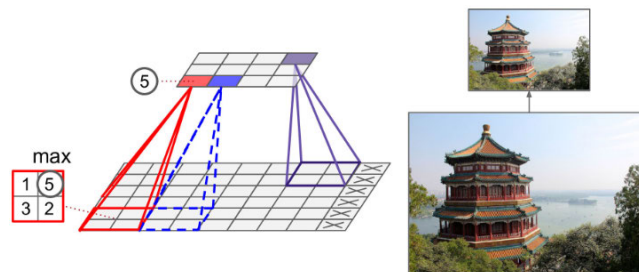


Figure 5: Maxpooling with a kernel size and stride of 2 (Géron, 2023)

This emphasizes the most prominent features, making the network more resilient to small variations and noise. When an image is slightly shifted, the max pooling layer still identifies the key features, and as a result, the overall representation remains similar. This property, known as translation invariance, ensures the model can detect features across various regions of an image consistently (Géron, 2023).

Local Receptive Fields Instead of connecting every neuron to all input pixels, each neuron in a convolutional layer connects to a small region of the input image called the local receptive field. As the receptive field slides over the image using a specified stride, it constructs a map of activations that indicates detected features' locations and strengths. For example, a neuron with a 5×5 receptive field in the first convolutional layer of a CNN processing a 28×28 image sees only 5×5 patches at a time. As it moves, the neuron processes information through its weights and biases to form a feature map, showing the position and strength of the detected features (Géron, 2023).

Shared Weights and Biases In convolutional layers, each filter (or kernel) has a set of shared weights and biases applied uniformly across the entire input. This means that the same filter is used at every location, enabling the detection of similar features, such as edges or textures, across different spatial regions. This also helps the network improve translation invariance (Géron, 2023), just like the pooling layers.

By using the same set of weights and biases across all spatial locations, the number of parameters required is significantly reduced compared to fully connected layers. This reduction not only improves memory efficiency but also decreases the computational load, allowing for faster training and inference (Géron, 2023).

Stride and Zero-Padding Stride and zero-padding define the relationship between input and output sizes in convolutional layers, Figure 6 shows the impact of different stride sides. Stride defines the step size of the sliding filter across the input image. A larger stride reduces the spatial dimensions of the output feature map by skipping positions, which speeds up computation but at the cost of detail. Zero-padding adds zeros around the image borders to preserve the feature map’s spatial dimensions and prevent information loss at the edges. Different padding configurations can change the output feature map size, balancing spatial resolution with computational efficiency. Zero-padding involves adding zeros around the image border, allowing filters to slide beyond the edges of the image and maintain the spatial size of the output feature maps (Géron, 2023).

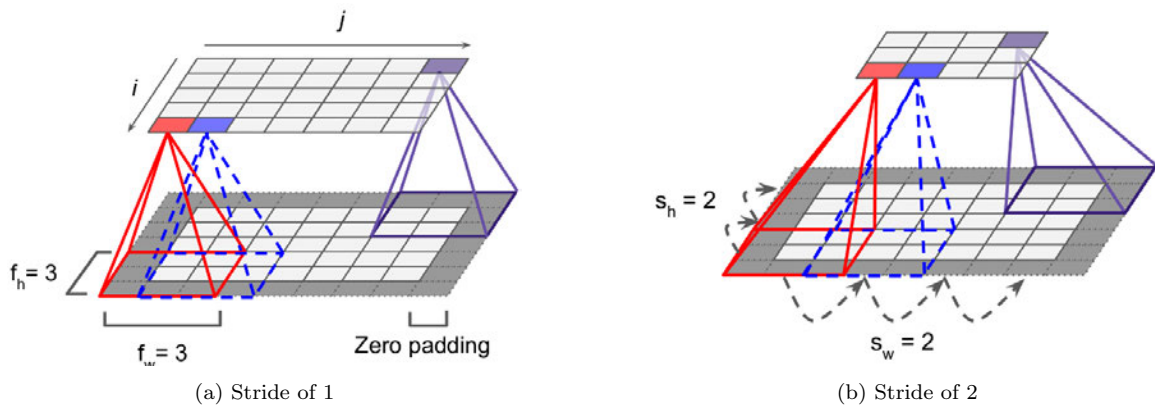


Figure 6: Effect of zero-padding and different stride sizes (Géron, 2023)

All these components work together to enable CNNs to perform exceptionally well in computer

vision tasks. Notably, CNN architecture is used as the backbone for state-of-the-art models like YOLO.

2.2.1 YOLOv8

Building upon CNNs, the YOLOv8 model by Ultralytics is a computer vision model that can perform various tasks including, object detection and instance segmentation. The Object Detection mode of YOLOv8 is designed to identify and delineate objects within an image by drawing rectangular bounding boxes around them. Conversely, the Instance Segmentation mode aims to precisely outline the full perimeter of each object, theoretically enabling a more detailed and accurate representation of their shapes and spatial relationships. However, this mode can inadvertently include parts of the background within the object outlines as well as exclude certain parts of the object, potentially compromising the performance of AD models as such models require highly consistent and accurately delineated training data to effectively distinguish between normal and anomalous instances. YOLOv8 is open-source and licensed under the AGPL-3.0 license, thus enabling both academic and commercial use (Ultralytics, 2024).

Architecture and Performance YOLOv8 employs an architecture built on a CNN-based backbone for feature extraction, a neck for feature fusion, and a head for output predictions, all processed in a single step, hence the name *You Only Look Once*. The backbone utilizes CSPDarknet, a common method for object detection systems (Ultralytics, 2024). The head uses several convolutional layers and a series of fully connected layers to output bounding boxes, object classifications, and confidence scores. YOLOv8 versions vary in size, such as YOLOv8n and YOLOv8l (Nano and Large), each providing a trade-off between speed and accuracy. For lightweight applications prioritizing speed, YOLOv8n is suitable. In scenarios requiring high precision, such as detailed image analysis, YOLOv8l offers superior accuracy (Ultralytics, 2024), and is the version applied in this paper.

Training and Fine-tuning The training of a YOLOv8 model usually starts with a pre-trained model, like YOLOv8l, which has been trained on the COCO dataset (Lin et al., 2021), allowing for the transfer of learned features which are generally applicable across a broad

range of objects and scenarios as this dataset includes a diverse array of everyday objects (Ultralytics, 2024). As such, the transfer learning is used to rapidly adapt to new, possibly domain-specific tasks through fine-tuning. In addition to layer freezing, this fine-tuning process involves modifying the output layer of the model to match the number of classes in the new task, resizing the final classification layer.

For validation, YOLOv8 uses intersection over union (IoU) method to measure the overlap between predicted and actual bounding boxes relative to the union of these boxes (Géron, 2023). An illustration of the IoU metric is depicted in Figure 7. YOLOv8’s ability to rapidly and accurately detect objects in complex environments makes it a good contender for the task of this paper (Ultralytics, 2024).

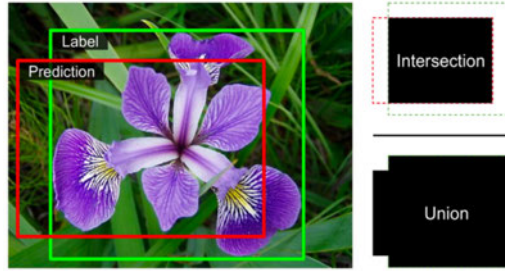


Figure 7: The IoU metric (Géron, 2023)

2.2.2 VGGNet

VGG19 is a CNN architecture proven to be effective not only in image classification but also in complex tasks involving feature extraction and image reconstruction (Simonyan & Zisserman, 2014).

Using VGG19 for Feature Extraction When comparing original and reconstructed images, such as those generated by CAEs (see Section 2.3), VGG19 can be used to extract high-dimensional feature representations of these images. By passing both the original and reconstructed images through the VGG19 model, one can obtain sets of feature maps for these images, typically from specific layers that are predetermined based on the desired complexity and depth of features.

For example, the first few layers of VGG19 can capture basic textures and edges, while deeper layers capture higher-level features that represent more abstract aspects of the images. By

extracting features from, say, the first and fifth convolutional layers, we can obtain a granular as well as a more holistic view of the visual content in the images (Simonyan & Zisserman, 2014).

2.3 Autoencoders

An autoencoder (AE) is a type of NN used to learn efficient representations of unlabeled data. Unlike other types of NNs, an AE is trained to reconstruct its own inputs instead of predicting a target variable. This characteristic allows AEs to capture important features of the input data in a compact form, making them useful for AD, denoising, and other applications where the model needs to capture the underlying structure of the data independently (Géron, 2023; Google Brain Team, 2024).

Architecture of Autoencoders An AE consists of two main components: the encoder and the decoder. The encoder compresses the input into a latent-space (also called embedding) representation, and the decoder reconstructs the input image from this representation. The goal is to minimize the reconstruction error, typically measured by MSE between the input and its reconstruction. The encoder part is a data compression algorithm which learns a compact representation of the input data. In a simple AE, the encoder could be a single layer or multiple layers of a NN. The decoder part aims to regenerate the input data from its encoded form with minimal loss. This part acts as a decompression algorithm, attempting to reconstruct the input data as accurately as possible from the compressed data created by the encoder (Géron, 2023).

2.3.1 Convolutional Autoencoders

A convolutional autoencoder (CAE) uses convolutional layers to compress and reconstruct images, combining the principles of CNNs with the encoder-decoder structure.

Architecture of Convolutional Autoencoders The encoder compresses an input image into a compact feature map using convolutional layers and max pooling, which reduces spatial dimensions while increasing depth. This compressed representation retains essential features of the input image, making it simpler for the decoder to reconstruct. The decoder then uses transpose convolutions (or deconvolutions) to gradually expand the feature maps back to the

original image size while reducing depth. It reverses the compression process using filters and stride, ensuring the final image retains key features and minimizes reconstruction loss. Ultimately, this symmetric design allows CAEs to reconstruct high-quality images that closely resemble the original input (Géron, 2023).

2.4 Imaging Tasks

Imaging tasks include preparation, analysis and interpretation of visual data to solve specific problems or assess image quality and features. The following concepts and techniques are relevant for this project, including image similarity assessment, image registration, and image classification.

2.4.1 Image Similarity

To compare the similarity of two images, a widely-used metric to assess image qualities is the Structural Similarity Index Measure (SSIM). The method examines changes in contrast, luminance, and structural information between the images, in order to provide a more perceptually based evaluation (Wang et al., 2004). This contrasts, for instance, MSE which considers the pure error between the images .

SSIM is based on the human visual system, which is highly adapted to extract structural information. Rather than only quantitatively comparing visible differences between images, the method considers changes in structural information, independent of light and contrast information. Then, all three components are evaluated by separate comparison functions and the results are combined into one overall metric. This enables SSIM to consider more than mere absolute errors and can serve as useful tool to assess consistency in image similarity between datasets in the context of this project. The index ranges from -1 to 1, where 1 indicates that the images are perfectly similar, 0 indicates no similarity, and -1 indicates perfect anti-similarity (Wang et al., 2004).

2.4.2 Image Registration

Image registration is a fundamental task in image processing. It involves aligning two images of the same scene or object, but from different viewpoints or by different cameras, effectively adjusting for differences in rotation, scale or skewness. This is essential for computer vision tasks, as adequate alignment of images enables more comprehensive and detailed analysis of actual differences in content, and as such is important for AD. Effective image registration is important for this project, since it alleviates modelling as greater emphasis can be focused on actual differences between the PCB products, and less on variance introduced by misalignments.

To achieve reliable image alignment, we employ Scale-Invariant Feature Transform (SIFT), a technique widely used due to its robustness in identifying and matching key points across different images (Lowe, 2004). This step is important for accurate subsequent analysis, as it significantly reduces the potential for error introduced by misalignment stemming from the data collection process, as discovered with the first data collection we conducted.

To enhance the SIFT method, the Random Sample Consensus (RANSAC) algorithm is implemented (Fischler & Bolles, 1981). The algorithm repeatedly selects a random subset of the image data, fits a model to these points and then determines how many other points fit this model well. The process continues until the algorithm finds the best fitting model (highest numbers in inliers) and can provide a reliable estimation also in cases where many outliers are present. RANSAC is widely adopted, especially for image registration, where improves image alignment by ignoring parts that does not match well.

The image registration process goes as follows:

1. **Find keypoints:** SIFT is used to identify and extracts keypoints from a reference image. The keypoints are features, e.g. corners and edges, which reliably can be identified across different images of the same scene.
2. **Describe keypoints:** SIFT describes each keypoint with a unique descriptor that is robust to changes in light, scale, and rotation. These descriptors can then be compared to find matches between different sets of keypoints and descriptors from another image.

3. **Filter matches:** After matching keypoints between images, there are usually some incorrect matches (outliers), if the images have significant differences or noise. RANSAC iteratively selects random subsets of the matches and uses them to estimate the transformation needed to align one image with the other. It evaluates the fitness of each estimated transformation based on how well it aligns with other matched keypoints.
4. **Selecting the Best Transformation:** RANSAC provides the transformation that has the highest number of inliers (matches that fit well with the transformation) which is then considered the best solution, ensuring that the final transformation is based on the most reliable keypoints.

By integrating RANSAC with SIFT (SIFT-RANSAC), one significantly improves the accuracy and reliability of image registration, even in conditions where there are many outliers or noise in the data. The combination is commonly used in various image applications, improving performance when precise image alignment is critical.

2.4.3 Image Classification

Classification in machine learning involves the identification and application of distinct features within a dataset to categorize new, unseen data. In binary classification scenarios, such as distinguishing between normal and defective industrial components, the outcomes can be presented using a confusion matrix that categorizes predictions into four groups: True Negatives (TN), False Negatives (FN), True Positives (TP), and False Positives (FP). Each group serves a specific role in the computation of various performance metrics, all which informs about a model’s performance trade-offs in different scenarios. Table 1 shows the confusion matrix design used to evaluate this paper’s models. For clarity, instead of the usual binary labels, *Negative* and *Positive*, this paper, respectively, uses *Good* and *Anomaly* to denote the outcome of the AD process.

	Predicted Good	Predicted Anomaly
Actual Good	True Negative	False Positive
Actual Anomaly	False Negative	True Positive

Table 1: Confusion Matrix as a Table

To assess individual model performance and to enable meaningful model comparisons, we use the following evaluation metrics; precision, recall, specificity, accuracy, and F1-score. It is important to achieve the right balance between these metrics is important, since prioritizing one metric too much over others can lead to poor model performance understanding and selection. Ultimately, this can lead to e.g. unnecessary increase in inspection times (false positives) or the amount of overlooked defects (false negatives), both undesirable in PCB manufacturing.

Precision Precision measures the accuracy of positive predictions and is the ratio of true positives to the sum of true and false positives. High precision indicates a low false positive rate.

$$\text{Precision} = \frac{TP}{TP + FP}$$

This reflects the model’s ability to correctly identify defects without raising false alarms, high precision means that when the model predicts a defect (e.g., a misoriented component, a tilted pin), there is a high likelihood that the defect truly exists. This reduces the potential for unnecessary rework or scrapping of boards, saving costs and time in the production process.

Recall Recall (also called sensitivity) is the proportion of actual positives that were correctly identified, and is the ratio of true positives to the sum of true positives and false negatives. A recall of 1.0 means that there are no false negatives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

It quantifies the model’s ability to detect all defect instances. A high recall value means that the system is effective at identifying defects when they are present, preventing defective PCBs from passing unnoticed. This is particularly important for defects causing product failures, such as incorrectly placed or oriented components.

Specificity Specificity is the proportion of actual negatives that were correctly identified as such, and it is the ratio of true negatives to the sum of true negatives and false positives. High

specificity indicates a low false positive rate.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

It reflects how well a model identifies non-defective items as defect-free. High specificity means minimal disruption of the work flow, where non-defective boards avoid unnecessary inspection. For ABC, high specificity minimizes costs related to false positives such as inspection time and delayed shipments.

Accuracy Accuracy measures the overall correctness of the model and is the ratio of the true predictions (both positives and negatives) to the total number of cases examined.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

A high accuracy indicates that a model can reliably differentiate between defective and non-defective units and may be trusted for quality control. However, accuracy may be a deceiving metric in cases when the classes being predicted are imbalanced. For example, given a dataset of 95% non-defect and 5% defect samples, a naive model that predicts all samples as non-defect achieves a 95% accuracy. This may suggest a well-performing model, but the metric does not inform on how well it discriminates between classes.

F1-Score The F1-Score is a harmonic mean of precision and recall, offering a balance between the two metrics. It is useful when the costs of false positives and false negatives are high or when classes are imbalanced. The F1-Score is calculated as follows:

$$F1 = 2 \times \frac{\text{Precision} \times \text{recall}}{\text{Precision} + \text{recall}}$$

It provides a more nuanced view of precision and recall of a model in one metric, informing on the ability to identify all relevant instances (high recall) correctly (high precision). A high F1-Score indicates that a model is both accurate and reliable in detecting defects without being overly conservative by marking too many non-defective items as defective.

Kernel Density Estimation Plot Kernel Density Estimation (KDE) plots are used to visualize the characteristics of a dataset, where KDE is a statistical technique for estimating the probability density of the data. Unlike histograms that categorize data into rigid, separate bins (categories), KDE provides a smoother distribution. Each data point is treated as the center of a small, bell-shaped curve, often Gaussian, which spreads out evenly to neighboring points. The curves are combined to form a continuous, overall estimate of the data distribution. This creates a fluid picture showing trends and patterns, helpful in detecting anomalies that deviate from the expected distribution (Waskom & the Seaborn development team, n.d.). As an example, we can establish a baseline distribution of expected error scores for normal PCBs by applying KDE to a model’s outputs after feeding it a set of good PCB images. When new images are fed to the model, the new error scores can be visually compared against the baseline with KDE plots.

3 Literature Review

This section presents the diverse field and nature of AD, factors underpinning the transition towards machine learning, and a summary of relevant academic material which guided the development of this project and the necessary considerations for practical applications.

3.1 Anomaly Detection

AD involves identifying unusual patterns that deviate from expected behaviors in data. These anomalies are variously described across different domains as outliers, exceptions, defects, and more, indicating their broad applicability and significance in diverse fields (Chandola et al., 2007). Identifying significant deviations early on is of great interest in a wide range of applications, particularly in manufacturing and production quality control systems.

3.1.1 Inherent Challenges

Integral challenges in AD lies in its complexity and the nuances involved in defining what constitutes normal versus abnormal instances. Two fundamental issues contribute to the complexity; 1) the boundary between normal and abnormal is often not easily defined, leading to potential

misclassifications of normal instances as anomalies and vice versa; 2) what is normal now may not be in the future, making the task of defining a fixed "normal region" problematic. These difficulties are compounded by variations in the definition of outliers across different contexts, each influenced by specific domain requirements and constraints. According to Chandola et al. (2007), these factors make it challenging to develop a standard approach to AD.

Moreover, the development of effective outlier detection systems is often made more challenging by practical issues such as the scarcity of labeled data for training models and the presence of noise mimicking true outliers. These factors often necessitate a certain level of tailored approach to the outlier detection problem, depending on the specific characteristics of the data and the nature of the expected anomalies. Consequently, as Chandola et al. (2007) point out, the literature on outlier detection is vast and varied, with methodologies ranging from statistical techniques to machine learning and deep learning (the use of NNs (Géron, 2023)), each adapted to meet the unique challenges of the application domain in question.

3.1.2 Transition to Deep Learning

In recent years, NNs have gained significant research interest within machine learning due to their exceptional performance across a wide array of applications in various fields. Before the integration of advanced machine learning techniques, AOI systems relied heavily on simpler, rule-based algorithms that focused on template matching and basic statistical methods. These systems were primarily designed to detect explicit flaws such as misalignment, missing components, and solder defects on PCBs by comparing the inspected items with a predefined template (Mirapeix et al., 2007). A major limitation of such methods was their inability to adapt to new or unseen variations without extensive reprogramming and manual adjustment (Abellan-Nebot & Subirón, 2010). In contrast, while a subset of machine learning, NNs both outperform traditional machine learning methods and exhibits increased adaptability, favoring it for AD tasks (Chalapathy & Chawla, 2019).

In their literature review on NNs and AD, Chalapathy and Chawla (2019) summarized the increased application of deep learning as a response to the following key limitations observed with traditional algorithms:

- **Performance:** Traditional methods often underperform in detecting outliers within complex datasets, such as images or sequences.
- **Scalability:** The need for AD at a large scale becomes critical as data volumes increases, to which traditional methods struggle to effectively scale up to.
- **Feature Learning:** NNs are better suited to autonomously learn hierarchical, discriminative features from data, reducing the need for manually crafting features. This facilitates end-to-end solutions that processes raw input data in various domains.
- **Ambiguous Boundaries:** The distinction between normal and anomaly is not always clear and can continually evolve across different data domains. This ambiguity presents challenges not just for traditional methods but also for NNs approaches. However, the latter facilitates continuous adaption more easily.

3.2 Deep Learning Approaches for Anomaly Detection

The main approaches to AD include supervised, semi-supervised, and unsupervised training strategies and selection of corresponding NN models, where the approach and model selection is primarily dependent on the extent to which using labelled data is feasible.

3.2.1 Overarching Strategies

Supervised This approach requires fully labeled datasets where both normal and anomalous instances are tagged. These methods are less commonly used in AD due to the scarcity of labeled anomalies and potential bias from class imbalance, given the often rare nature of anomalies. While supervised techniques tend to be highly accurate when labeled data is abundant, they are less used in label-scarce environments due to practical limitations, e.g. encapsulating sufficient scenarios of anomalies with labels (Chalapathy & Chawla, 2019; De Albuquerque Filho et al., 2022).

Unsupervised Unsupervised methods, on the other hand, does not rely on any labels at all, making it suitable for situations when anomalies are not only rare but also completely undefined. This approach assumes that normal samples significantly outnumbers anomalies, enabling a model to learn the general characteristics of normality from the majority of the

data. The distinction from semi-supervised is the lack of explicit curating of data to guide the learning process, requiring the model to implicitly define what constitutes normal and abnormal behavior based on the input data characteristics (Chalapathy & Chawla, 2019; De Albuquerque Filho et al., 2022).

Semi-Supervised Semi-supervised AD only uses labeled data from normal instances, and operates with the assumption that these labels accurately represent what normal conditions are. This method is useful in scenarios where normal behavior is well-understood and anomalies are rare or difficult to label due to their scarcity or unknown characteristics. In semi-supervised learning, a model is explicitly trained to recognize that the labeled normal data is representative of non-anomalous cases. Any deviation from the learned norm, which the model has not been explicitly taught to recognize as normal, is then flagged as an anomaly (De Albuquerque Filho et al., 2022).

An essential idea related to the not supervised paradigm is the concept of one-class classification, training models on one class of data, on the assumption that it is possible to encapsulate a boundary around the normal data, and data points that deviate would be potential anomalies (Chalapathy & Chawla, 2019).

This approach is inherently suited for situations where the 'normal' class is well-defined and abundant, while the 'anomalous' class is rare or not well-defined for possible anomaly instances, like in our case:

- **Scarcity of Anomalies:** Since anomalies are rare, collecting a representative dataset of all possible anomalies is impractical. One-class models circumvent this by focusing on modeling what is normal.
- **High Cost of Defects:** Defects in manufacturing are costly. A model that can effectively identify deviations from normalcy without needing overly-extensive examples of defects can prevent costly production errors.
- **Simplicity and Efficiency:** One-class models are often simpler and more computationally efficient compared to traditional models and do not need to learn multiple classes.
- **Adaptability:** These models can adapt to changes in what is considered 'normal' without

requiring a complete retraining of the system. This is particularly beneficial in PCB manufacturing where updates to component designs or configurations might occur.

3.2.2 One-Class Models

Models trained with a reconstruction objective represent some of the earliest and most prevalent NN strategies for AD (Ruff et al., 2021), and recurring deep learning modelling techniques revolving around the one-class idea for image-based AD include CNNs and variants of AEs such as CAEs, variational autoencoders (VAEs), and adversarial autoencoders (AAEs) (Chalapathy & Chawla, 2019; De Albuquerque Filho et al., 2022; Schneider et al., 2022). Even though only the CNN and the CAE architectures are explored in this paper, respectively as a baseline and the main model architecture, VAEs and AAEs hold potential for future exploration, especially in cases that would benefit from their higher complexity.

CNNs CNNs are generally the more simple of the recurring techniques for AD, with one, not so simple, example being Hamdi et al. (2020), who demonstrate the application of deep one-class learning for AD in video streams, highlighting the adaptability of CNNs in various operational contexts. As for classifying anomalies, Ruff et al. (2018) introduces a method that encapsulates data within a minimal hypersphere in the feature space, optimizing for a compact representation of normality, which inspired us to create a CNN architecture that aims at reducing output vectors towards a zero vector given a *good* input. By training CNNs to minimize MSE to a zero vector, they can effectively identify anomalies as deviations from this target. Further, this method is supported by the framework discussed by Chalapathy et al. (2019), which explores the distinctions between normal and anomalous patterns through learned representations.

Convolutional AEs CAEs can perform AD tasks due to their ability to learn a compressed latent representation of images. By training exclusively on normal instances, one can create a baseline of what constitutes normal behavior, which should result in low reconstruction errors for similar, *good*, images and significantly higher reconstruction errors for anomalies, making the reconstruction error an indicator for identifying unusual patterns. A comprehensive study by Schneider et al. (2022) compares different types of AEs for AD and identifies CAEs as a

strong candidate due to their simplicity and relatively low computational cost, in which CAEs demonstrated high performance on simpler datasets like MNIST (LeCun et al., 1998), where normal patterns are well-defined and reconstruction errors provide clear differentiation between normal and anomalous images.

In Schneider et al. (2022)’s work, however, CAEs also proved to be limited in their capacity to handle complex datasets such as CIFAR-10 (Krizhevsky, 2009), which involve more intricate image structures and variability. Despite this indications of limitations with regard to image complexity, de Oliveira et al. (2023) used CAEs successfully in a practical PCB AD system by specializing the architecture for the nuances of PCB components, while training multiple, separate, models on different regions of input images. The architecture proposed includes layers specifically designed to handle the nuances of PCB imagery, incorporating elements like max pooling to reduce dimensionality and convolutional transpose layers to reconstruct the input image from its encoded latent space representation. Their combination of pixel-wise and perceptual loss evaluation improved AD despite diverse environmental conditions. A similar method was proposed in Bergmann et al. (2019), who argue that relying on MSE loss alone can result in inconsistent results with even slight placement inaccuracies. With this, de Oliveira et al. (2023)’s system shows high effectiveness in less well-controlled environments, where the images are not expected to line up perfectly with previous data, making it suitable for, say, situations when a technician needs to manually inspect electronics (de Oliveira et al., 2023). Further, they used one-class learning, amongst other reasons due to the limited availability of training datasets and the unsupervised nature of AEs in learning relevant features. Although their focus was on AD across considerably varying image angles and lighting conditions, the study’s goal aligns closely with the objectives of our project. As such, de Oliveira et al. (2023)’s paper inspired us to implement a similar architecture in this paper.

Variational AEs VAEs can be used for AD due to their ability to model complex data distributions through probabilistic inference. Unlike standard AEs that compress input data into deterministic latent representations, VAEs learn a probabilistic latent space characterized by a Gaussian distribution, defined by its mean and standard deviation. Furthermore, this also allows VAEs to generate new samples similar to the training data by sampling from this

learned distribution. Such a framework works for AD as anomalies often manifest as outliers that diverge statistically significantly from the normal distribution (Schneider et al., 2022).

The work by Schneider et al. (2022) emphasizes that VAEs can excel in AD tasks involving complex datasets, as. In their study, VAEs demonstrated superior performance on the CIFAR-10 dataset. They achieved this by capturing nuanced patterns and deviations in the latent space that simpler AEs could not fully discern. Further validating this, An and Cho (2015) applied VAEs for AD in cybersecurity, demonstrating that their probabilistic nature enabled effective identification of network attacks that deviated from expected patterns. However, training VAEs requires careful tuning of hyperparameters like the learning rate and the weight of the VAE-specific loss method. If not properly configured, this can result in blurry reconstructions, alongside the higher computational demands compared to CAEs. However, the use of VAEs has proven effective for AD in complex scenarios. For instance, Chen et al. (2020) introduced an unsupervised approach using a sliding-window convolutional variational autoencoder to detect spatial and temporal anomalies in industrial robots. This architecture processes multivariate time series data, enabling real-time AD. By applying similar principles to PCB components, our models could identify even minor defects that deviate from the expected distribution, ensuring high accuracy and sensitivity in detecting subtle abnormalities. The potential use of VAEs for ABC is discussed in Section 7.4.1.

Adversarial AEs AAEs, introduced by Makhzani et al. (2016), merge the ideas behind VAEs and generative adversarial networks (GAN) to manage the structure of the learned latent space. In simpler terms, the latent space is like a compact code representing the most important features of the input data. By employing a secondary adversarial network (inspired by GANs), AAEs enforce a specific statistical distribution on this latent space, typically a Gaussian distribution. This means that the encoded representations of the input data are adjusted to fit this target pattern. Consequently, the decoder, which is responsible for transforming this compact code back into a full image, can create samples that closely match the original input. Additionally, the adversarial training process refines this latent space, making it more organized and less entangled. This regularization ensures that the latent representations are meaningful and distinct, which should help the model better understand and reproduce the key features of the

input data.

Schneider et al. (2022) observed that AAEs can generate a more consistent and structured latent space than VAEs, which results in clearer reconstruction of normal data and thus potentially improved AD performance. Goto et al. (2020) explored the use of AAEs in inspecting solder joints on PCBs by using adversarial regularization to establish a structured latent space, providing a clear distinction between normal and faulty solder joints. Their results showed that this approach significantly reduced false positives, improving detection even when only a small number of anomalous samples were available. Similarly, Son et al. (2020) used an AAE-inspired setup with one AE as the generator and another AE as the discriminator for a GAN system, which minimized false positives in their testing with MNIST, CIFAR-10, and a brain tumor dataset.

However, Schneider et al. (2022) found that while AAEs do offer promising results, they are the most complex and time-consuming of the AEs to train. The adversarial training process requires careful tuning of the GAN component, which can significantly increase training complexity and time. For example, they noted that training the discriminator network to distinguish between real and generated representations adds heavily to the overall computational cost. Further, in their experiments, the AAEs were often outperformed by simpler CAEs as well as VAEs. Thus, although AAEs seem advantageous in some contexts, the complexity and training time make them less practical compared to other AE types, and are as such not discussed further in this paper.

3.2.3 Large Multimodal Models

Going a step beyond AEs, Large Multimodal Models (LMMs) offer another approach to AD. Alongside the concurrent craze around Large Language Models like GPT-4 (OpenAI et al., 2024), its LMM sibling GPT-4V (OpenAI, 2023) can efficiently process, among others, image and video data. By leveraging prompts like class information, expert annotations, and reference images, GPT-4V can detect global and fine-grained patterns with minimal training data. Cao et al. (2024) highlight the model’s ability to distinguish between normal and abnormal samples with zero or few-shot learning, that is, having zero to only a handful of reference im-

ages, enabling it to provide comprehensive explanations for its decisions. Furthermore, (Zhang et al., 2023) propose a framework called GPT-4V-AD that applies GPT-4V to AD. This framework uses granular region division, prompt design, and segmentation to evaluate the model’s effectiveness quantitatively and qualitatively. Their experiments show that GPT-4V achieves competitive results in zero-shot AD. However, given current limitations, simpler models with more transparent architectures are preferred for this project. Just as with the VAEs, the potential implications for ABC using LMMs for AD are discussed later, in Section 7.4.2.

3.2.4 Supervised YOLO-based Models

Even though YOLO is only one part of our proposed system, with a comprehensive dataset including potential anomalies, a supervised approach, as shown by Beak et al. (2023), can turn a YOLO model into an AD method on its own. In their study on cosmetic manufacturing, they propose a labeling methodology that achieves high inspection accuracy for various defect types using the YOLOv7 architecture. ABC could consider using this method if they are willing to invest the time and effort to create labeled datasets for each PCB board design, which could be especially applicable for boards that receive recurring orders annually. However, due to the labelling requirements, this method is also not explored further.

3.2.5 Selected Approaches

In this project, we adhere to a strategy of starting with minimal complexity. Thus, given the nature of our dataset, primarily non-defect images, pursued a semi-supervised approach focusing on CAEs and CNNs as the primary architectures for AD in hand-mounted PCB components.

Convolutional Autoencoders CAEs are deemed appropriate in this context because their encoder-decoder architecture allows them to capture component-specific characteristics autonomously while being relatively simple to train. By training each CAE on exclusively normal instances of regions of interest (ROI), the model learns a highly specialized baseline, yielding low reconstruction errors for normal samples and significantly higher errors for anomalies. This specialization ensures a high degree of precision, particularly important in PCB manufacturing, where even slight defects can lead to significant issues. Furthermore, CAEs are computationally efficient, making them suitable for training on the relatively small datasets available for each

component type. As such, the architecture’s simplicity and adaptability suit ABC’s proof-of-concept well, allowing rapid training and evaluation without the need for overly complex hyperparameter tuning.

Convolutional Neural Networks As a baseline, CNNs offer strong feature extraction capabilities due to their hierarchical structure, enabling them to capture relevant patterns and distinguish between normal and defective instances. Their discriminative nature makes them suitable for detecting component anomalies where specific characteristics, such as tilt or incorrect mounting direction, are present, as shown by Hamdi et al. (2020). By minimizing MSE to a target zero vector for normal instances, CNNs can effectively identify deviations as anomalies.

4 Methodology

This section delineates the methodology employed in this thesis, integrating both theoretical and practical frameworks to achieve a POC that lives up to ABCs requirements of cost-efficiency and specialization. The methodological structure is adapted from the Research Onion model and the Cross Industry Standard Process for Data Mining (CRISP-DM) framework.

4.1 Research Framework

This paper uses the Research Onion model (Saunders et al., 2023), illustrated in Figure 8, as it provides a detailed framework for developing a comprehensive research methodology. Each layer of the onion represents a step in the research process, guiding the selection of proper methods and techniques to achieve our objectives.

4.1.1 Research Philosophy

This research adopts a pragmatist philosophy, which is ideal for addressing real-world challenges with practical solutions. By starting with a clear problem, as entailed by our research question, and focusing on practical results, this project aligns with the pragmatist emphasis on actionable knowledge. This ensures that the outcomes are not only theoretically sound but also practically useful, providing tangible benefits to the industry.

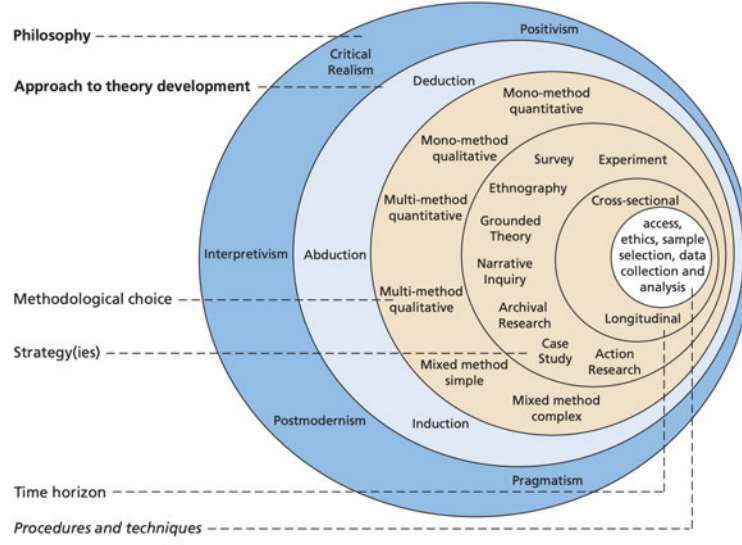


Figure 8: The Research Onion

4.1.2 Research Approach

The research employs an inductive approach, as it builds from empirical observations towards broader generalizations and theories. The research begins with data collection from AD experiments using NNs, specifically CNNs and CAEs, which are known for their effectiveness in similar applications. These models are applied to a dataset of PCB images to evaluate their performance in detecting anomalies. The findings from these experiments are used to develop a framework that enhances understanding of the applicability of these NN models in AD. This approach allows for the derivation of theories from observed data, under the guidance of the CRISP-DM framework, which systematically organizes data analysis and enhances the empirical grounding of the research findings.

4.1.3 Methodological Research Choice

The research utilizes a mixed-methods design, integrating quantitative and qualitative aspects. Quantitatively, the project leverages extensive image analysis and model performance metrics, such as accuracy, precision, recall, and F1-Scores, to evaluate the effectiveness of various approaches in detecting anomalies. Qualitatively, the emphasis is placed on the system’s applicability and usability within an operational context. For instance, the implementation of a color-coded visualization system categorizes the severity of anomalies, ensuring the system’s practical utility by presenting complex information in an accessible format, facilitating an easy

decision-making process for PCB inspection personnel. Furthermore, a straightforward retraining protocol accommodates new PCB designs ensures the system’s adaptability and long-term viability in a manufacturing environment that sees many different product designs, like the case of ABC.

4.1.4 Research Strategy

In alignment with the CRISP-DM framework, the research strategy of this project is experimental, focusing on systematic development, testing, and evaluation of machine learning models within a controlled setup. The project employs a detailed experimental setup that allows for precise control over variables such as lighting, angle of image capture, and the physical positioning of PCBs.

4.1.5 Research Time Horizon

As detailed by Saunders et al. (2023), research time horizons can be classified into two types: longitudinal, which observes changes over time, and cross-sectional, which assesses a phenomenon at a specific point in time. This project employs a cross-sectional study design, which is suited well to the objectives of enhancing quality control for hand-placed components in PCB manufacturing using AOI. This approach facilitates the evaluation of computer vision techniques on a fixed dataset of PCB images, providing insights applicable not only to current manufacturing challenges but also adaptable to future changes in PCB designs. This setup ensures that the developed models can be immediately tested and validated under consistent conditions, enabling potential scalability and utility in both ongoing and future applications.

4.1.6 Techniques and Procedures

The final layer of the Research Onion concerns the specific data collection techniques and analytical procedures used in this research. Data collection involves capturing images from stable camera setups designed to accommodate different types of PCBs. Analytical procedures are extensively covered under the CRISP-DM framework, detailed in the subsequent section, which outlines the stages from data understanding and preparation through to modeling, evaluation, and deployment.

4.2 CRISP-DM Framework

This thesis uses the CRISP-DM framework by Chapman (2000) which facilitates the systematic development of data-driven solutions by delineating a sequence of phases that can iterate as necessary. This framework is depicted in Figure 9, which illustrates the cyclic nature of a data project and its adaptive progression through each phase.



Figure 9: CRISP-DM Process Model (Chapman, 2000)

4.3 Business Understanding

The Business Understanding phase is foundational, establishing the project’s objectives, the problem scope, and the potential impact of the solution.

The project’s deliverables were shaped through discussions with ABC, with the primary objective being to explore the use of computer vision for AD in hand-mounted components, culminating in a POC. Anything beyond the POC was welcomed by the company, but not part of the deliverable requirements. However, upon achieving promising results, we extended the scope of the project beyond the POC to not only meet MGE’s immediate needs but also to evaluate the adaptability of the solution across different PCB designs and production scales. The business understanding from this phase ensured that the POC would align well with the operational realities and economic constraints of PCB manufacturing, emphasizing the need for a cost- and time-effective, robust, and adaptable solution.

4.4 Data Understanding

The next phase in the CRISP-DM process is Data Understanding. This phase begins with data collection and continues with activities aimed at understanding the data. It involves identifying data quality issues, gaining preliminary insights, and uncovering interesting data subsets (Chapman, 2000).

As such, the type, nature, and quality of data that the project will utilize are analyzed. In this project, the focus is on understanding the complexities of the image data captured from PCBs which are vital for training the AD models. Detailed insights into the data collection setup, the characteristics of the captured images, and preliminary data exploration are key activities of this phase.

The initial data collection setup was inspired by the promising results of de Oliveira et al. (2023), who investigated the application of computer vision for AD in an uncontrolled environment with substantial variance in image quality. Subsequent modelling, however, revealed that the initial data collection procedure using a rudimentary camera setup did not yield usable results. As such, the project’s final data collection method utilized a more stable camera setup to capture high-quality images of various PCBs under consistent lighting and angles, essential for the subsequent training of computer vision models and performance of the final system.

4.4.1 Data Collection

This section details the technical configurations and iterative adjustments made to the data collection setup, emphasizing the rationale behind each enhancement to optimize the quality of the images captured. The data collection process was conducted in two stages, which involved an iterative approach to refine and enhance the system based on initial modelling outcomes. Upon analysing the reasons for poor results from modelling of the initial batch of images, a revision and rebuild of the data collection system was determined necessary. In both setups a Canon 600D with its standard kit lens, the Canon EF-S 18-55mm f/3.5-5.6 III, is used. The 600D has an APSC crop sensor, which is smaller than a standard full-frame sensor, resulting in a 1.6x amplification of the image, meaning that the effective focal length of the lens is actually 28-88mm.

Experimental Setup The first setup comprised of a black plastic box, with reflective walls and ceiling, having the following internal dimensions: 26 centimeters wide, 36 centimeters long, and 30 centimeters tall.

Elevated PCB Placeholder The PCB was placed on a white plate outfitted with small protruding pins to secure the PCB in a fixed position. The white plate was supported by a metal plate contoured into a reversed U-shape at the bottom of the box. This elevated the PCB roughly 10-15 centimetres.

Two Sources of Light Two strong light sources were positioned at the far ends of the box. To mitigate direct glare onto the PCB, a plate was placed in front of each source, at an angle of 45 degrees, to divert the light upwards and to the sides. The box's ceiling was lined with white paper to disperse light more evenly, whereas the box's walls remained unaltered.

Camera The camera was positioned on top of the box's lid, inserted through a hole fitted to the width of the lens. To support the camera from sideways movements, three L-shaped metal brackets were tightly attached to the lid. In order to secure the camera in position, the camera was pressed inside the plates to exert pressure on the sides of the camera to achieve stability. Considering the height of the box, the distance between the PCB and the camera was approximately 15-20 centimetres. A remote control was used to capture an image to minimize vibrations.

Challenges with the Initial Setup

1. The lid had to be lifted to take the PCBs in and out, with the camera still attached.
2. The absence of continuous camera charging in place necessitated frequent camera removal.
3. A short distance between the object and the camera necessitated a wide angle, 18mm in this case, to capture the objective, adding distortion.
4. Uneven light distribution causing variability in the illumination of different PCB areas.

Frequent removal of both the lid and camera introduced variability in the camera's placement and angle, exacerbating the natural variance in components' appearance. This led to significant variations within what would typically be considered non-defect products across all images,

ultimately affecting the accuracy of defect detection models, often resulting in false positives.

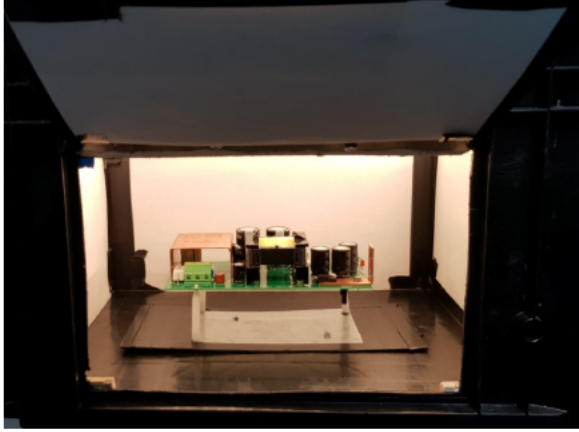
Revised setup To ensure greater consistency in data quality and enhance model performance, the data collection system underwent significant modifications.



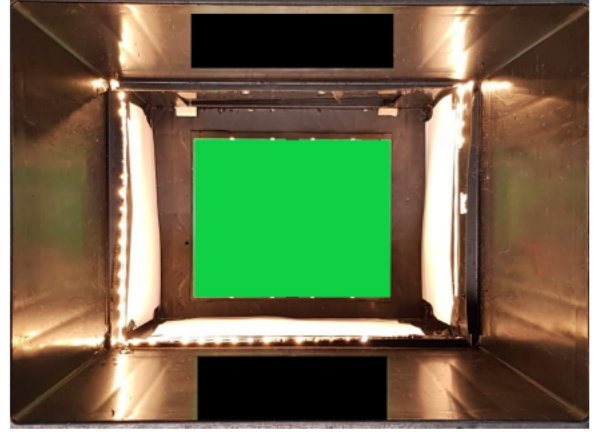
Figure 10: The revised system

Lowered PCB Placeholder A metal plate with more appropriately sized pins, larger and thicker, ensures a fixed positioning of the PCBs, reducing positional variance through a tighter fit between the pins and the PCB. Moreover, by using a flat metal plate, coated with black tape in the final stage, the object is further away from the camera lens.

Uniform Light Source Rather than having two sources of light, a uniform light source from all directions is preferred. Thus, LED strips are attached to each wall with a neutral color (figure 11b portrays a warmer light than what is the case, due the camera used for documentation). Moreover, the light source is placed higher up, as getting the light diodes further away from the PCB helps avoid reflection. As seen in figure 11b the plastic walls of the black box produces



(a) Side view of the main box



(b) Top-down view, PCB hidden per ABC request

Figure 11: Side- and inside view of setup during construction

sharper reflection than white paper. Therefore, all sides, including the hatch, is lined with paper to further distribute the light more evenly.

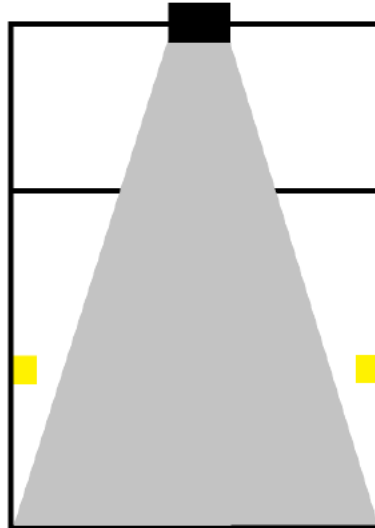


Figure 12: Geometry relations between perceptive field, size of hole and positioning of lights

Next, to prevent the camera to be directly exposed to the lights, the size of the opening carved out the in the bottom of the box which the camera is resting on, was carefully considered with regard to the positioning of the lights. Figure 12 illustrates the geometrical considerations necessary between the camera's (the black square at the top) perceptive field (gray area) and the light (yellow).

Camera A new support mechanism to fix the camera in place is seen in Figure 13. Rather than relying on squeezing the camera into position, without adequate measures of metrics to ensure

the exact same position every time, a more reliable system is employed. Supporting planks are placed across the top lid, with ridges to prevent side to side movement. To prevent forward or backward movements, metal brackets are fastened to the planks and the lid, which can be adjusted. On the opposing support plank from where the base of the camera is attached, a hinge supports the camera roof. The height of the hinge and angle of the camera is determined by screwing a screw on the inside higher or lower. However, with the current design, the camera still has to be taken in and out for battery charging, but the company has been advised to acquire a direct-to-wall solution to eliminate movements, which can be done using a cheap dummy battery.

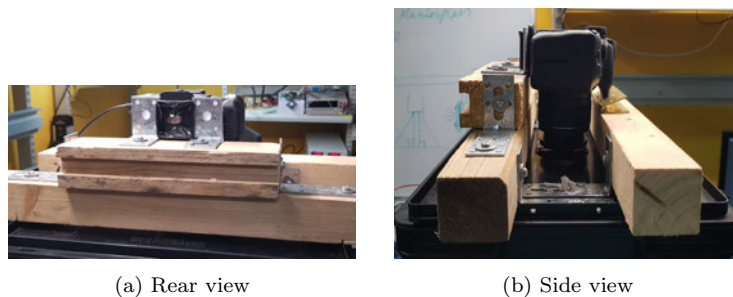


Figure 13: Overview of new system for camera positioning

Mitigating Image Distortions When capturing images, various factors contribute to distortions that affect the quality and accuracy of the photographs. One such distortion is barrel distortion, which occurs when using wide-angle lenses, such as our lens at the 18mm focal length setting. Barrel distortion causes straight lines to appear curved outward towards the edges of the image, leading to a subtle yet noticeable distortion effect. To mitigate barrel distortion, adjusting the focal length to a higher value, such as 55mm in our case, helps significantly.

In addition to barrel distortion, another prevalent image distortion is chromatic aberration, which shows up as unwanted colors along edges, often appearing purple-like. Chromatic aberration is particularly prominent when using a low aperture, which refers to the size of the opening where the lens lets in light. Cameras on automatic modes tend to prefer lower aperture, as this lets in more light, reducing the need for compensation, which typically introduces noise or other unwanted effects. The severity of chromatic aberration varies from lens to lens, and cheap kit lenses, like ours, need high apertures to eliminate this distortion, and as such, we adjusted the aperture to $f/16$, which lets in about 5% the amount of light when compared

to this lens' maximum aperture, f/3.5, as seen in this formula: $\left(\frac{3.5}{16}\right)^2 \approx \frac{1}{\left(\frac{16}{3.5}\right)^2} \approx \frac{1}{20.41} \approx 4.9\%$, underscoring the need for proper light sources.

Another obvious benefit of higher aperture values in the context of PCB photography is that higher values allow more depth to be in focus at any time, which is important when components have varying heights. Furthermore, shooting at focal lengths near the extremities of the lens, such as 18mm, can sometimes exacerbate chromatic aberration. Therefore, careful consideration of both aperture and focal length settings is essential to ensure a high image quality.

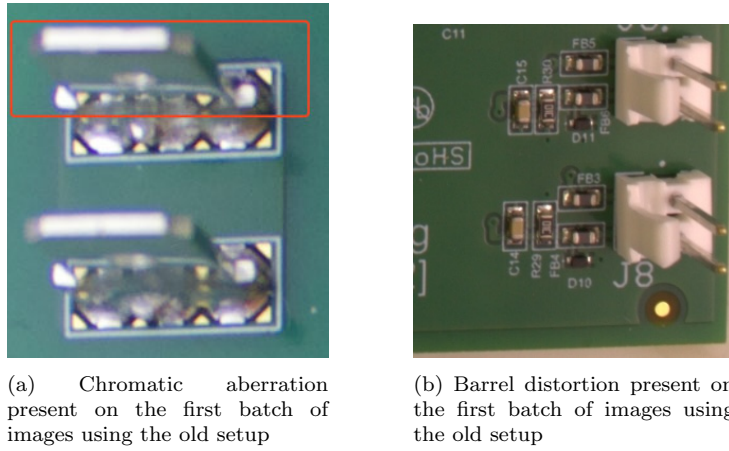


Figure 14: Chromatic aberration and barrel distortion seen in the first batch of images

Optimizing Distance Between Object and Camera The new design stacks three boxes on top of each other, which are held tightly in place with screws and nuts, and when combined with the lowered PCB placeholder, result in an increased distance between the object and the camera. By increasing the distance between the camera and the object, the lens can now be set to a focal length of 55mm, effectively eliminating the barrel distortion seen with the wider focal lengths.

This arrangement not only mitigates distortions but also ensures that the captured images maintain clarity, detail, and consistency.

Practical Use Ease-of-use has improved significantly, as the insertion of products is now done sideways instead of top-down. A hole and hatch is installed on the side of the base box, which facilitates further reduction of vibrations and movements to the camera. The hatch is covered with a flexible lining to prevent external light pollution. Additionally, image capturing

is now performed through the native software of the camera provider, allowing live viewing which provides a more user-friendly system while also allowing for faster processing for each PCB.

Results The redesigned setup streamlined operations and significantly reduced variability in the images captured. This was quantitatively confirmed through the use of SSIM.

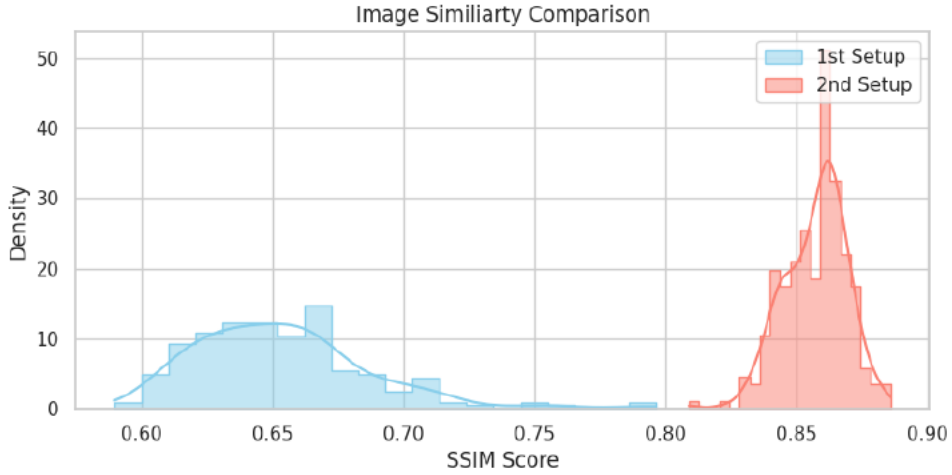


Figure 15: SSIM evaluation of consistency between images within each dataset

A random image was taken from the old and new dataset and the SSIM score against the remaining images were computed. Figure 15 shows a significant increase in image similarity on average and reduction of intra-variance for the new dataset. Considering that the camera was removed three times for charging during the last data collection, the plot indicates that even higher consistency may be achieved with further adjustments to the system, e.g. not removing the camera for charging. For an example of the implications imperfect training data, see Section 7.2.2.

The final hardware configuration used in-house materials available at ABC, supplemented with commercially available lighting and minor fastening equipment, resulting in direct costs approximated to €20.

4.4.2 Data Description

In total, the dataset consists of 254 images, where 224 images are of good products and 30 images contain one or more anomaly. Figure 16 displays the raw image of a good PCB, which

has a width and height of 5184 x 3456 pixels, 72 dots per inch, a bit depth of 24, and using the sRGB color channel. All images were captured at f/16, an exposure time of 1.3 seconds, ISO-100, and an effective focal length of 88mm. The resulting images take up 4.5 megabytes.

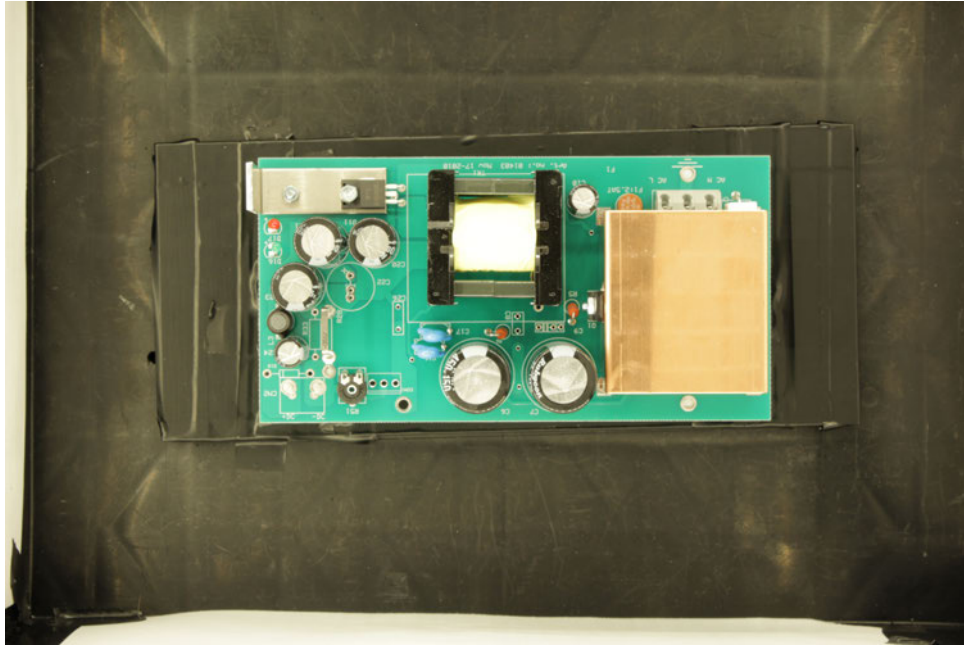


Figure 16: Raw image

4.5 Data Preparation

Following the CRISP-DM framework, the next step is Data Preparation. This phase involves transforming raw data into a format suitable for modeling. This section will outline these methods in detail, ensuring the data is optimally prepared for effective training of the computer vision models.

4.5.1 Data Formatting and Initial Preprocessing

The preparation of the data to a uniform format before modelling is necessary, but also depends on the type of models to be applied. To ensure consistency, all data is first preprocessed into a common format before subsequent preparation to accommodate specific ROI extractions, modelling needs and experimentation. First, all images, with and without anomalies, are aligned by applying SIFT-RANSAC. In addition to performing image registration, the images are rotated to the same direction and downsized to a width and height of 2709 x 1395 pixels, and a file size of approximately 600 Kb. Figure 17 portrays the base format.

The final dataset was then split into a training set with 199 images, a validation set of 25 images, and a test set comprising 30 images of anomaly-ridden PCBs. Within the subsequent modelling methods, the test images are segmented into either grids or specific components, ensuring the presence of both defect-free and defective instances within each ROI, allowing the evaluation of the model’s ability to accurately detect and classify anomalies across various subsections of the PCB.

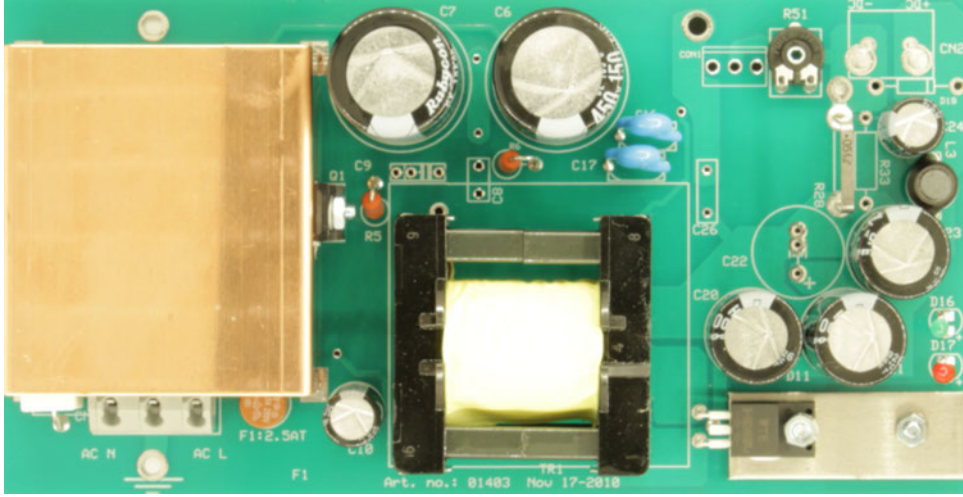


Figure 17: Base format after initial preprocessing

Three approaches to ROI extraction are evaluated in this paper. The first is a grid-wise method, where the image is systematically partitioned into smaller sections, facilitating the creation of the GP-CAE and GM-CAE models. The second and third approaches are component-specific: one utilizes a template-based method for ROI extraction based on static coordinates, and the other employs dynamic ROI cropping with a finetuned YOLO model. This results in the development of the TC-CNN, YOLO-CNN, TC-CAE, and YOLO-CAE models. All models will be discussed in further detail (see Sections 4.6 and 5.1). The component-specific approaches all use the base format after initial preprocessing (see Fig. 17) as inputs, as either the template or YOLO application automatically performs the cropping. As such, these models require no further preprocessing, unlike the Grid Models.

4.5.2 Partitioning for Grid Models

In the grid-wise approach, the image is divided into grids prior to modelling as to enable concentrated scrutiny while at the same potentially enabling global AD (unknown errors outside of known potential error ROIs). Moreover, another rationale for segmenting a larger image into smaller regions, often equally sized in width and height, is that this approach isolates the impact of errors, particularly when multiple anomalies are present. When errors are multiple and spread across a larger area, they become proportionally less noticeable, but each error may nevertheless be of equal importance. Thus, utilizing grids rather than the whole image, the differences between the input image and reconstructed image from separately trained AEs become more pronounced, aiding in discovering anomalies.

de Oliveira et al. (2023) follows a strategy of dividing an image into equally sized grids as a precaution to preserved spatial dimensions. Additionally, the grid-size used are relatively small, both with regard to computational cost and to allow for granular scrutiny of details. We chose a grid size of 920x720 pixels for two primary reasons:

1. **Content:** Different regions of the current PCB exhibit varying levels of natural variation, such as the yellow tape in the middle of figure 18 (grid 2, 2), which shows considerable variations in the good image dataset. By maintaining a moderately large grid size, we attempt minimize the impact of this region relative to the overall analysis.
2. **Efficiency:** While the system is unlabelled compared to the YOLO system, the bad dataset still needs to be split manually according to whether the grid contains an anomaly. Thus, fewer but larger grids streamline the categorization process. With our dimensions, we also ensure some overlap, to capture potential discrepancies missed by non-overlapping partitioning. Currently, there are 30 images, and the chosen dimensions results in 6 grids, meaning that 180 images have to be manually processed. In contrast, smaller grids sizes, for instance 512x512 or 256x256 pixels, increases to 18 or 66 and between 540 and 1980 images to be manually processed, respectively, necessitating a cost-time versus performance analysis for the specific PCB product type.

The PCB analysed in this paper is considerably smaller than several other PCB products the

company would be interested in employing an AOI system, but which are unavailable due to logistics. Given the rapid increase in manual processing for more granular examination of this relatively simple PCB, we therefore investigate the potential of medium-sized grids, with the benefits and consequences of this entails.

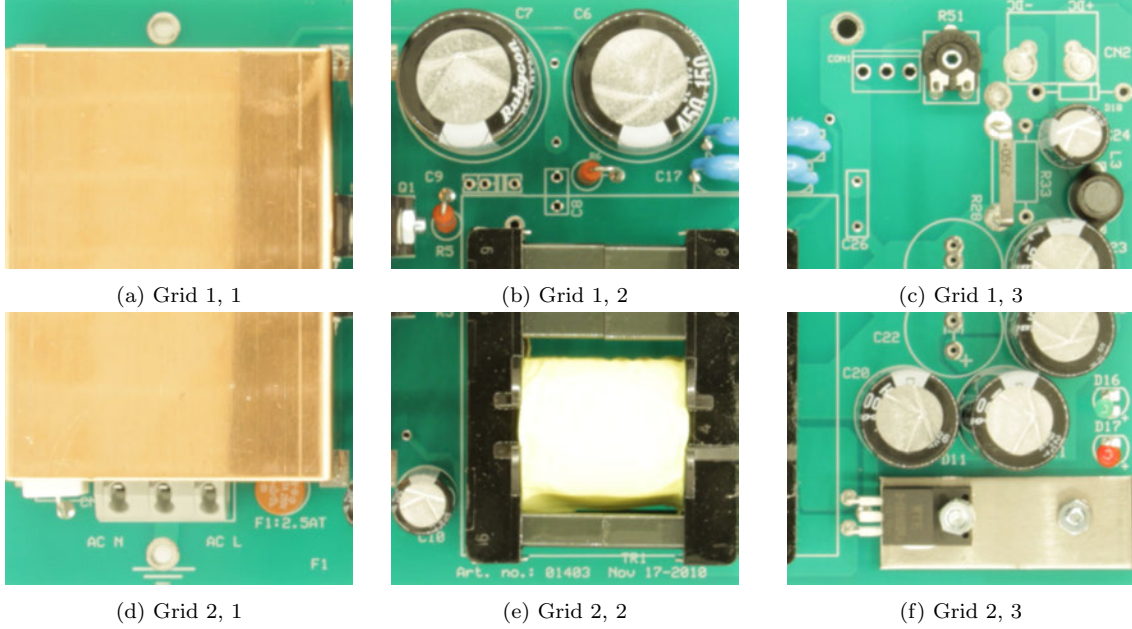


Figure 18: Partitioning of image into 6 regions

4.6 Data Modelling

Continuing with the CRISP-DM framework, the Modeling phase comes next. In this stage, the prepared data is utilized to develop and train models capable of detecting anomalies in PCBs. This section will discuss the selection of the applied model architectures, detail the training procedures, parameter tuning, and describe the iterative refinement process of the models based on the feedback from the evaluation phase.

After extracting the described grids into training, validation, and test sets, explored two CAE implementations were explored to encapsulate general representations of the good PCBs; a CAE with a pixel loss function and a CAE combining pixel- and content loss. This yielded insights that shaped the investigation of the component-specific approaches to revolve purely around models using pixel loss functions.

4.6.1 CAE Architecture

For the CAE architecture, we implemented a similar architecture as used by de Oliveira et al. (2023), displayed in Table 2. We use an input dimension of 256×256 pixels, as to balance providing a high enough resolution to capture details while considering that the number of features, equal to $\text{width} \times \text{height} \times \text{color channels}$, grows exponentially to growth in image dimensions.

Encoder	Output
Conv + BatchNormalization + LeakyReLU	$128 \times 128 \times 32$
Conv + BatchNormalization + LeakyReLU	$64 \times 64 \times 64$
Conv + BatchNormalization + LeakyReLU	$32 \times 32 \times 128$
Conv + BatchNormalization + LeakyReLU	$16 \times 16 \times 128$
Conv + BatchNormalization + LeakyReLU	$8 \times 8 \times 256$
Conv + BatchNormalization + LeakyReLU	$4 \times 4 \times 256$
Conv + BatchNormalization + LeakyReLU	$2 \times 2 \times 256$
Fully Connected + BatchNormalization + LeakyReLU	1024
Fully Connected + LeakyReLU	500
Decoder	Output
Fully Connected + BatchNormalization + LeakyReLU	1024
TranspConv + BatchNormalization + LeakyReLU	$4 \times 4 \times 256$
TranspConv + BatchNormalization + LeakyReLU	$8 \times 8 \times 256$
TranspConv + BatchNormalization + LeakyReLU	$16 \times 16 \times 128$
TranspConv + BatchNormalization + LeakyReLU	$32 \times 32 \times 128$
TranspConv + BatchNormalization + LeakyReLU	$64 \times 64 \times 64$
TranspConv + BatchNormalization + LeakyReLU	$128 \times 128 \times 32$
TranspConv + Sigmoid	$256 \times 256 \times 3$

Table 2: Architecture of the CAE

Kernel size The size of the kernel is set to 5×5 , as to preserve enough information to accurately reconstruct the inputs. Just like de Oliveira et al. (2023), the kernel size stays the same throughout the network. However, one could also vary this if, say, smaller details are not captured well enough, as was done in Kim et al. (2021), who attempted to detect finer PCB anomalies. As such, they used both 3×3 and 5×5 kernel sizes.

Filter size The CAE architecture employs a series of convolutional layers with progressively larger filter sizes. For instance, the first convolutional layer outputs a feature map with dimensions $128 \times 128 \times 32$, meaning it reduces the spatial size of the original 256×256 image by half

while producing 32 feature maps that highlight specific patterns. The next convolutional layers continue this pattern, with the second layer producing 64 feature maps of size 64×64 , further reducing the spatial dimensions while expanding the depth to capture more diverse patterns. The final convolutional layer in the encoder yields a feature map of $2 \times 2 \times 256$, vastly reducing the image dimensions but maximizing the feature depth to provide a highly compressed yet information-rich representation. The decoder reverses this process by expanding the feature maps back to the original dimensions using transpose convolutional layers (denoted *Transp-Conv*). For instance, the first TranspConv layer in the decoder outputs a feature map of $4 \times 4 \times 256$, gradually reconstructing the original image by doubling the spatial dimensions while maintaining the depth of 256 feature maps. The final layer produces the fully reconstructed image at $256 \times 256 \times 3$, representing the original resolution with all three color channels intact.

A stride of 2 is used for each layer, reducing the spatial dimensions of the a layer’s output by roughly half, decreasing the amount of computations in subsequent layers while increasing the field of view of each filter. The padding is set to ‘same’ to add zeros around the edges of the input and allow the convolution operations to be applied at the borders of the image. Effectively, this preserves information that would otherwise be lost or undervalued at the periphery of the image.

Activation Function The use of Leaky ReLU activation functions helps prevent the ”dying neurons” issue commonly associated with standard ReLU activation, as evidenced by comparative studies (He et al., 2015; Xu et al., 2015). Batch normalization is integrated throughout the layers to accelerate training and improves generalization (Ioffe & Szegedy, 2015).

Combining Pixel and Perceptual Loss for the CAE This modelling approach is inspired by the work of de Oliveira et al. (2023), who demonstrated promising results in detecting and segmenting modifications in images of assembled PCBs using AEs with the same architecture as outlined in Table 2 together with the combined MSE and Perceptual loss function (see 4.6.1). If one uses the content loss alone, it would result in only structural information to be considered between images, potentially neglecting smaller details and texture dissimilarities of importance. In their study, this combination proved effective in enhancing a model’s AD

capabilities in highly diverse imaging conditions, such as shifts in camera angles or lighting discrepancies. Our decision to experiment with blending both types of losses was influenced by their successful application and the inherent variability in our initial dataset, as described in the end of Section 4.4.1. Our experimental design for the grid-wise approach therefore integrates both MSE and perceptual loss in the same manner as de Oliveira et al. (2023).

To extract features, we used VGG19. The initial layers of VGG19 capture basic structures like object edges, whereas the deeper layers produce increasingly abstract representations of the image features. The appropriate layers to extract relevant features therefore depends on the problem at hand. de Oliveira et al. (2023) arrived at the best performance through empirical testing of both the selection of layers and the balance between perceptual and pixel-wise loss, and in their case, a higher weighting on the perceptual component yielded the best performance along with a combination of early and mid-level layers.

Thus, we conducted empirical experiments to identify the most effective layers of VGG19 for feature extraction and the weighting of these features, in addition to the layer configuration found most effective by de Oliveira et al. (2023). Considering the more controlled nature of our data collection, which featured less variance in lighting and angles in addition to anomalies taking both smaller and larger shapes, we explored several configurations, presented in table 3. We trained separate models for each grid for each layer configuration outlined, adjusting the weights between MSE and perceptual loss (95% and 99%), and included a baseline model that utilized only MSE for comparative analysis.

Configuration	Layers	Weights
Early	[3, 8, 17]	[.95, .99]
Middle	[5, 8, 13, 18]	[.95, .99]
Late	[17, 26, 35]	[.95, .99]
All	[3, 8, 17, 26, 35]	[.95, .99]
No	None	[0.0]

Table 3: Layer configurations for feature extraction and weighting during loss calculation tested (‘Middle’ is the same as the one used by de Oliveira et al. (2023))

4.6.2 CNN Architecture

The baseline model for the component-instance approach involves using a simple yet effective CNN architecture. The architecture comprises three convolutional layers, each followed by max pooling. The first layer uses 32 filters, the second 64, and the third 128, with each filter sized at 3×3 . This configuration is chosen to incrementally capture more complex and abstract features of the images as we move deeper into the network. Each convolutional layer employs a standard ReLU activation function.

Layer	Output
Input Image	$64 \times 64 \times 3$
Conv + ReLU + MaxPooling	$32 \times 32 \times 32$
Conv + ReLU + MaxPooling	$16 \times 16 \times 64$
Conv + ReLU + MaxPooling	$8 \times 8 \times 128$
Flatten	8192
Dense + ReLU	128
Dense + Linear	64

Table 4: CNN architecture for the component-instance approach

After processing through the convolutional and max pooling layers, the architecture transitions to a flattening step followed by two dense layers. The first dense layer consists of 128 neurons with ReLU activation to continue processing the features while maintaining non-linearity, ensuring that no information valuable for detecting anomalies is lost. The final dense layer utilizes a linear activation function to produce a 64-dimensional output vector. The model weights are tuned to output this as a zero vector given a *good* input, makes it easier to spot any deviations that might indicate defects, as the vector from these should contain values markedly different from 0.

Kernel size The kernel size chosen for the convolutional layers is 3×3 as this size is meant to balance the model’s ability to capture fine details and textures, which are necessary to spot subtle anomalies in PCB components. A larger kernel, like 7×7 , would significantly increase the number of parameters, making the model computationally heavier and more prone to overfitting, especially given the relatively small size of our dataset. Conversely, 1×1 kernels, while effective for adjusting the depth of the feature maps and interacting across channels, would likely fail to capture the necessary spatial information needed for detecting specific anomalies

such as misalignments and incorrect placements of components. As such a kernel size of 3×3 is used as it allows the model to learn effectively with fewer parameters. Further, this kernel size is used for CNNs in papers with similar goals, such as Hamdi et al. (2020) and Liu and Wang (2023).

Filter size As for the progression of filters, 32, 64, to 128 in successive convolutional layers, the idea is to incrementally increase the network’s capacity to capture more complex and abstract features as we delve deeper into the network. The initial layer with 32 filters captures basic aspects of the images, like edges and contours. As the layers progress, the increasing number of filters allows for the detection of more complex features, such as the orientation of components or subtle defects that could be missed by simpler models. This hierarchical increase in filters ensures that each layer builds upon the knowledge accumulated from the previous, refining the detection capabilities of the network.

Activation Function Traditional ReLU is chosen because it introduces non-linearity to the network while promoting sparse representations, effectively setting all negative activations to zero. This sparsity aids in amplifying deviations from the learned patterns during training, making ReLU particularly advantageous for AD. While attempts were made with Leaky ReLU, it was observed that it didn’t yield improvements in results, even though it should theoretically address the dying ReLU problem.

The final activation function in the output layer is linear rather than one that squashes outputs, such as sigmoid. By using a linear activation, the model can produce a 64-dimensional output vector where anomalies are more apparent due to the direct numeric deviations from a zero-vector baseline expected for normal data. The linear activation ensures that the network can detect even slight deviations without limiting the output to a fixed range, thereby emphasizing and revealing anomalies clearly.

The Constant Zero Vector as the Target In the architecture, the constant zero vector serves as a unique target, distinguishing this approach from typical supervised learning tasks. Unlike traditional methods where models predict outputs closely aligning with provided labels, this one-class model treats all normal data instances aiming to align their outputs to a zero

vector. This zero vector, composed entirely of zeros and having a dimensionality equal to the number of neurons in the output layer (64 in this case), acts as a symbol of 'normality' in the model's learning process.

Training to Minimize Deviations The model's training involves fine-tuning the weights to minimize the MSE between the predicted outputs and the zero vector. The MSE quantifies the average of the squares of the differences between predicted and target values, providing a clear measure of the model's performance in replicating the zero output for normal conditions. This training strategy is key to enabling the model to identify anomalies as deviations from the zero vector lead to higher MSE values.

4.6.3 Training Setup

All modelling experimentation were conducted on the UCloud platform, an online environment for cloud computing, on an NVIDIA H100 GPU combined with 48 threads of an AMD EPYC 9454 CPU and 192 GB RAM. While other options for cloud computing include services such as AWS (Amazon) and Colab (Google), we opted for UCloud due to receiving free computing hours through our university. The models were programmed in Python using TensorFlow, a library providing an ecosystem for end-to-end solutions for machine learning and NNs (Google Brain Team, 2024). All models (including the fine-tuned YOLO model) were trained using Adam (see Section 2.1.4), as the optimizing algorithm with an initial learning rate of 0.001 for 200 epochs. As for loss functions, all component-wise models use MSE, while the grid-wise models use either purely MSE or a combination between MSE and perceptual loss. With this top-of-the-line hardware, training time for each individual model was between 5 and 10 minutes. The total computing cost, including coding, experimenting, and modelling time landed at approximately €200, using UCloud's internal cost system. Table 5 shows the various models categorized by their cropping method and base architecture.

4.7 Evaluation of Models

The next component of the CRISP-DM process is Evaluation. This phase assesses the performance of the developed models against predefined metrics. This section will, given the

Category	Model	Crop Method	Loss Function	Base Model
Grid-wise	GP-CAE	Grid	Perceptual + MSE	CAE
	GM-CAE		MSE	CAE
Component-wise	TC-CNN	Template	MSE	CNN
	TC-CAE		MSE	CAE
	YOLO-CNN	YOLO	MSE	CNN
	YOLO-CAE		MSE	CAE

Table 5: Model Overview: Categories, models, crop methods, loss functions, and base models

exploratory nature of our investigation, examine the applicability of two different approaches to solve the problem, as model performances is computed differently within each domain.

4.7.1 Grid Models

In total, the layer and weight configurations outlined in Table 3 amounts to nine combinations to assess. For each configuration, the six grid-models were trained, each model was then fed every corresponding grid image in the train and validation dataset, one at a time, to output respective reconstructions errors. The distribution of these errors constitute the expected range for good images. From these distributions, the mean and standard deviation were computed, along with thresholds values set to equal one, two, three, and four standard deviations away from the mean for each grid model. Subsequently, all grid models were fed the test dataset and if the reconstruction error exceed any of the threshold values, an anomaly was registered. All classifications made by the grid models were then aggregated per model configuration, since every grid is equally important, and for each threshold value.

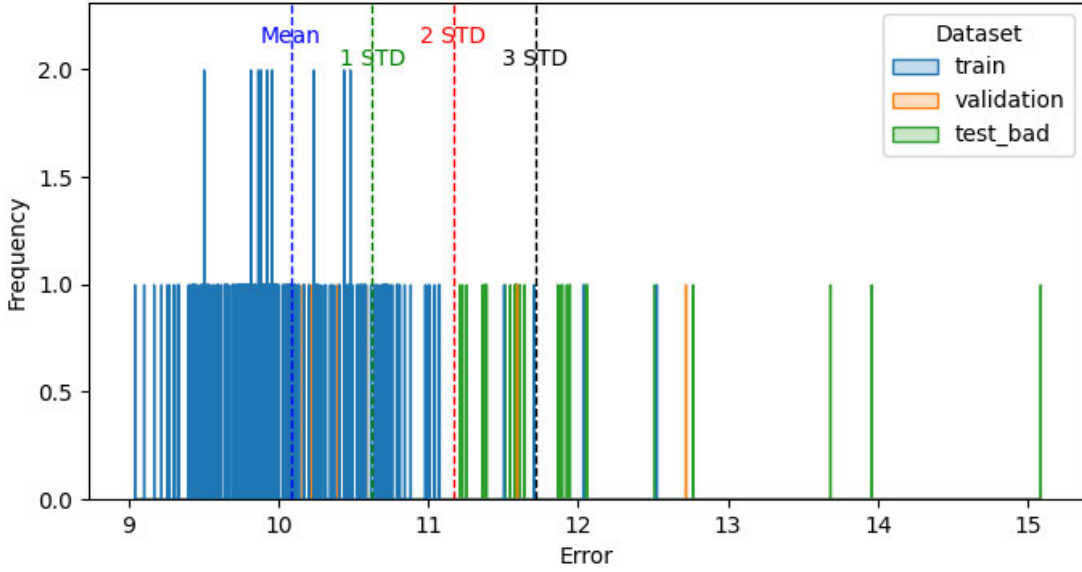


Figure 19: Reconstruction error distribution illustration for a particular grid model

Figure 19 illustrates the error distribution for a single grid, for which a new image’s reconstruction error would be compared to. The distribution of the normal reconstruction errors varies both between the different model configurations and internally between grids, which is why we evaluated the various models at different threshold level.

4.7.2 Component-Instance Models

The evaluation of the Component-Instance Models differs depending on the architecture of the NN. While the CNN-based models use the output vector norm to determine anomalies CAE-based models use reconstruction errors.

The CNN-based Models As the CNN-based models are trained to minimize the MSE between the predicted outputs and the zero vector, the model weights are adjusted such that the output for all training samples (which are considered ‘normal’) is as close as possible to this zero vector. Conversely, an anomalous input, which the model has not encountered in the training phase or significantly deviates from the training samples, should result in a larger output vector norm. Thus, the magnitude of this output vector becomes a metric for AD, where a higher norm suggests a potential anomaly. When the model encounters new test images, the

output vector’s magnitude is calculated using the Euclidean norm, serving as an anomaly score:

$$\text{Anomaly Score} = \|\text{features}\|_2$$

where $\|\cdot\|_2$ denotes the Euclidean norm, and *features* represents the model’s output, and as any significant deviation in the input from the normative pattern (learned during training) induces larger activations in the dense layers, the output vector’s norm increases.

The CAE-based Models To dynamically adapt the threshold for AD, the CAE-base component-instance models use the reconstruction error of a validation set (containing only *good* images) to establish baseline anomaly thresholds. The threshold is then set at one standard deviation above the mean of these scores, allowing the model to adjust to slight variations in different batches or changes in normal conditions without manual recalibration.

5 Results

In this section, we present and compare the performance of different models developed as part of our component-instance approach. Our analysis spans several model categories, each tailored to different aspects of AD in PCBs. The models were tested across a range of metrics, including accuracy, precision, recall, specificity, and F1-Score, to gauge their performance.

5.1 Model Performance Overview

After experimenting with various models and configurations, the relevant models within each model category are presented. Table 6 displays the respective scores in each evaluation metric. The full overview of all model performances can be found in the Appendix (Tables A1 and A2).

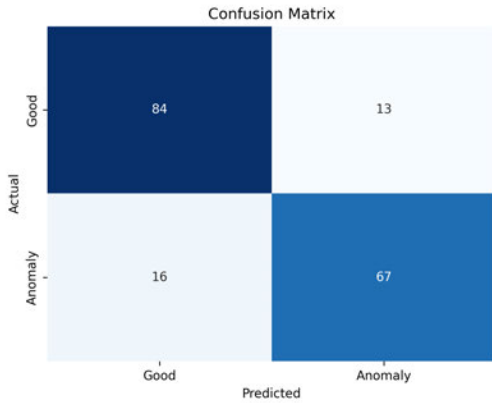
The test dataset consists of 30 images of PCBs with one or more defect per board. For the grid-wise approach, this entails that the models were tested on a total of 180 grid-images, with a ratio of 83 images with one or more anomaly present, and 97 anomaly-free images. For the template and YOLO approaches, the same test dataset translates into 244 cropped images of good components and 146 images of anomalies.

Category	Model	Metrics				
		Accuracy	Precision	Recall	Specificity	F1-Score
Grid-wise	GM-CAE	.84	.84	.81	.87	.82
	GP-CAE	.86	.88	.81	.91	.84
Component-wise	TC-CNN	.83	.72	.86	.80	.78
	TC-CAE	.92	.83	1	.88	.91
	YOLO-CNN	.89	.83	.82	.92	.82
	YOLO-CAE	.99	.98	1	.99	.99

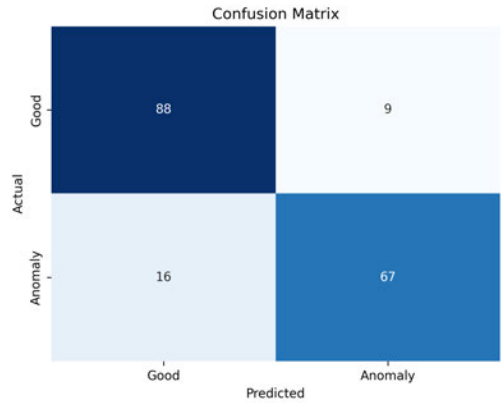
Table 6: Performance Overview: The best performing model in each category (marked in bold) versus respective baseline models

5.2 Grid-Wise Models

The grid-wise approach yields overall inferior performance compared to the component-specific approaches. The GM-CAE performs on par with the baseline model for best performing category, dynamic cropping, whereas the GP-CAE merely surpasses the same baseline model in terms of F1-Score by two percentage points, 82% vs 84%, in stark contrast to the overall best model, YOLO-CAE. Both grid models achieve very similar true positives/negatives, with a substantial false classifications of both grid images that are good and those containing anomalies, as seen in Figure 20, relative to the best models.



(a) GM-CAE: Considering only pixel differences



(b) GP-CAE: Considering pixel and perceptual differences

Figure 20: Comparison of grid models

While both models achieve the same recall score, the perceptual model does outperform the pixel focused model in terms of precision and specificity, albeit not by much. Even though the grid approach performs surprisingly well, considering the increased area size considered in as opposed to the component specific methods, a recall of 0.81 and precision of 0.88 would nevertheless lead to a substantial amount of missed anomalies and false alarms.

5.3 Template-Cropping Models

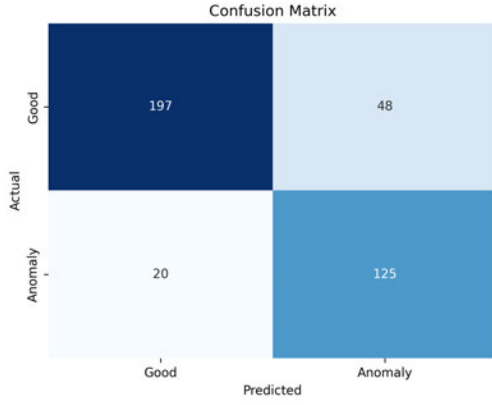
The transition from the grid-wise approach to component-specific models begins with the exploration of the template-cropping method. This method involves using predefined coordinates to isolate specific component instances from all images within the dataset. Such a targeted approach should allow for precise training and application of the models to these isolated regions, potentially enhancing the model’s ability to detect anomalies due to the consistent framing of the component of interest.

5.3.1 TC-CNN Model

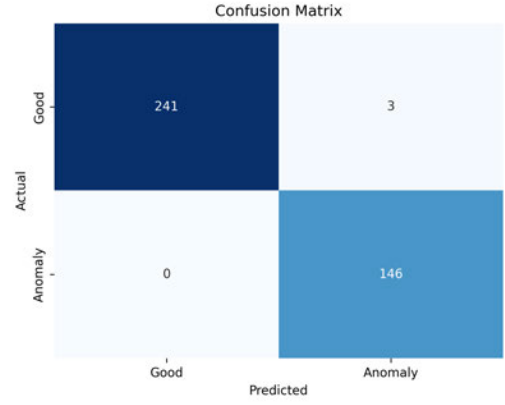
The Template Cropping with Convolutional Neural Networks (TC-CNN) model is the baseline for the TC method. As illustrated by the confusion matrix in Figure 21a, the recall of the TC-CNN model, at 0.86, indicates a poor effectiveness in detecting true anomalies, an important aspect for ensuring that few defects pass undetected. Further, the F1-Score of 0.78 underpins the need for improvement in distinguishing between normal and anomalous conditions more accurately. This rate of false positives would lead to many unnecessary follow-up inspections.

5.3.2 TC-CAE Model

The Template Cropping with Convolutional Autoencoders (TC-CAE) method improves upon the TC-CNN model, yet shows weaknesses. The confusion matrix in Figure 21b indicates an absence of false negatives, suggesting that the model captures all actual positive cases (that is, anomalies), giving it a recall of 1. However, the presence of significant false positives impacts the model’s precision, landing it at 0.83, indicating that while the model detects all anomalies, it also mistakenly identifies normal instances as anomalies. The specificity score of 0.88, however, suggests that while the model is good at identifying anomalies, it also needs to be improved



(a) TC-CNN: Template cropping with CNN



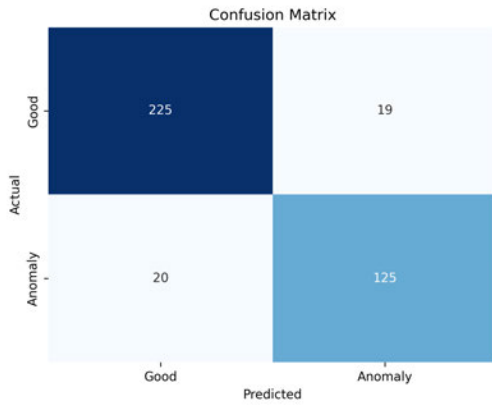
(b) TC-CAE: Template cropping with CAE

Figure 21: Comparison of template models

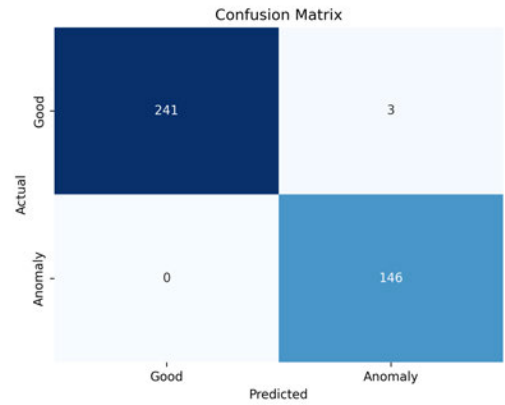
in terms of reducing false alarms. The F1-Score of 0.91 confirms the model’s robustness, while leaving room for further refinement to balance the recall and precision more effectively.

5.4 YOLO-Cropping Models

Following the evaluation of the template-cropping models, the more sophisticated YOLO-based cropping models are used to more accurately pinpoint, isolate, and crop component-instances within images, which should improve the accuracy and adaptability of the AD process.



(a) YOLO-CNN: YOLO cropping with CNN



(b) YOLO-CAE: YOLO cropping with CAE

Figure 22: Comparison of YOLO models

5.4.1 YOLO-CNN Model

The YOLOv8-based Cropping with Convolutional Neural Networks (YOLO-CNN) model, as shown in Figure 22a, demonstrates a decent foundation with an accuracy of 0.89 and a precision

of 0.83. However, the model suffers from a high number of false negatives, with 20 out of 145 true anomalies missed. This leads to a recall of 0.82, which is not ideal for high-stakes environments such as PCB production where missing an anomaly can have severe implications. The F1-Score of 0.82 further reflect the model’s need for improvement in consistently identifying true anomalies without increasing false detections.

5.4.2 YOLO-CAE Model

Improvements are evident with the YOLO-based Cropping with Convolutional Autoencoders (YOLO-CAE) model, where precision reaches a high 0.98. This high precision ensures that nearly all positive detections by the model are valid, reducing unnecessary interventions. Just like the TC-CAE, the YOLO-CAE model achieves a recall score of 1, indicating no missed true positives and thus maintaining operational safety by catching every potential defect.

The specificity of the YOLO-CAE model is 0.99, vastly outperforming the TC-CAE, suggesting a significant reduction in false positives. This robustness in correctly identifying non-anomalous instances allows the model to operate more efficiently within a production line, minimizing disruptions. The F1-Score of 0.99 further underscores the model’s superior performance across all metrics.

Comparing the YOLO-CAE to the earlier TC-CAE model, it is clear that the YOLO-CAE’s tighter cropping mechanism contributes to its higher performance metrics. The superior F1-Score of the YOLO-CAE model, in particular, highlights its balanced precision and recall, making it the preferred choice for implementation in the AD pipeline. As such, this model is selected as the primary model for further evaluation, as detailed in the subsequent sections.

6 Deployment and Application

The final phase of the CRISP-DM framework is Deployment. Although primarily theoretical in an academic context, this section will discuss hypothetical deployment scenarios and provide a guideline for potential real-world application. As such, this section details the practical application of the YOLO-CAE method. It encompasses the fine-tuning of the YOLO model,

the training and deployment of component-instance models, and the methods used for AD and visualization.

6.1 Model Preparation and Training

The initial phase of the application involves fine-tuning a pretrained YOLOv8 model, which will serve as the cornerstone for the component-instance detection and cropping, which in turn will be used to train individual component-instance-specific CAEs. A flowchart of the training pipeline is shown in the Appendix in Figure A1.

6.1.1 YOLO Model Fine-tuning

Initially, the pipeline is set to either fine-tune a pretrained YOLOv8 model or utilize an already fine-tuned model. If no fine-tuned model is provided, the function proceeds to fine-tune a pretrained model by utilizing transfer learning (see Sections 2.1.6 and 2.2.1). This process starts by using a labelling method, like the online platforms CVAT or Roboflow, which are both free and simple to use. This project’s YOLO-CAE model is based on the Object Detection mode and tight bounding boxes that cover each entire component, even if adjacent components are partially included. Roboflow was preferred in this project due to the comprehensive nature of their dataset export after labelling, as the file structure after unzipping the downloaded dataset matches the exact input requirements of our training pipeline. Our proposed system requires that each type of component (such as *Small capacitor*, *Medium capacitor*, *Metal resistor*) gets its own class. Differentiating between each instance of that class is done automatically later in the process.

The pipeline’s augmentation settings and hyperparameters for the YOLO training are significantly adjusted from the default values, as these reflect a typical *non-static* use-case of the YOLO architecture. Due to the homogeneity inherent in single-board datasets and the fixed perspective of PCB photography, very few labelled data points are needed in this process, and as such, only six labeled images, split equally across train, test, and validation sets were used to fine tune the entire YOLO model.

The training pipeline will, if needed, automatically create the rest of the necessary directories

and files to set up a ready environment. A YOLO training configuration file is also present, and must be edited to include the correct path name of the root training folder. Upon successful training, or if using an existing fine-tuned model, the pipeline delineates directories for storing outputs, such as predicted labels, images, and cropped component images. These categorizations ensure a streamlined workflow for any dataset size and allow the model to automatically process and categorize of all component-instances on the PCB.

6.1.2 Cropping and Saving Component-Instance Images

In the next part of the pipeline, the YOLOv8 model is applied across all images within a given root directory, which should contain folders for train, validation, and test images. To correctly isolate each component-instance, a reference image is used to automatically define a baseline, consisting of a dictionary of each component and instance of that component. As such, if there are more instances of the same component, these will be separated into their own component-instance folder. This baseline dictionary includes the expected bounding box coordinates of the component-instances, enabling an IoU comparison to determine which component-instance has been found by the YOLOv8 model on a new input image. For each image, detected components are cropped and stored in their specified directories, alongside their respective labels and bounding box coordinates.

6.1.3 Component-Instance Model Training

Conditional on the training setting being enabled, the system initializes training for CAEs for each component-instance found in the previous step. The CAEs are trained specifically with the images from the 'training' directory. Each CAE then learns inner representations, or a baseline of what constitutes *normal* appearance for various components, against which test images are compared to identify defects. After each run of a component-instance, the model weights with the lowest loss are saved under a run-specific folder, with a name that includes the final loss of that model, such that the system can select the best-performing model for future tasks, in cases where there have been multiple training runs.

6.1.4 Model Training Evaluation

After the training of a given component-instance model, the pipeline can be set to reconstruct unseen images of that instance. These reconstructions and their errors will be output by the pipeline so they can be visually inspected, as seen in Figure 23.

A quick look at the reconstruction errors in Figure 23 indicates that anything over 0.0100 would likely be considered an anomaly, assuming which, one FP, the second from the right, is present. The specific component shown is *only* present on the first PCB dataset, and a close look will reveal that that there is a black background behind each component. This is because that specific run utilizes the Instance Segmentation mode of YOLOv8 to crop out only the component itself, and not the background. This means that the entire system *is* compatible with this cropping method too, but a close look shows the somewhat random inclusion of parts of the green background and exclusion of the tips of the pins, making the method inferior to the bounding boxes of the standard YOLOv8 Object Detection method. It does though avoid the problem of interference from other components, a problem which can be seen later in Figure 28. Furthermore, the poor quality and inconsistency of the first setup (see Section 4.4.1) is visible when comparing the 'original' images across the figure.

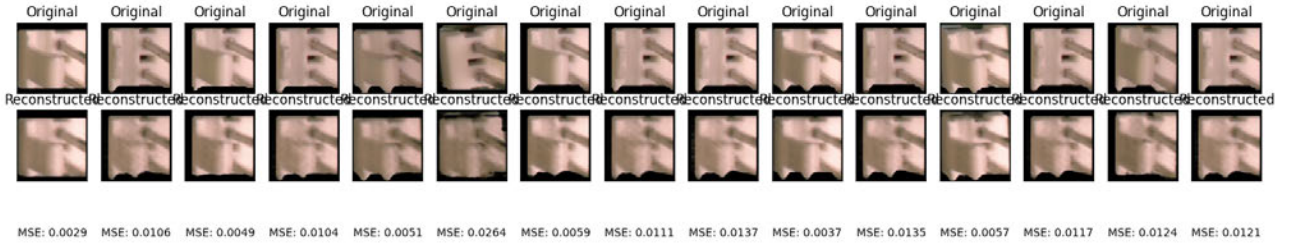


Figure 23: Reconstructions of unseen data by a newly trained CAE

If a CSV file containing the *true* labels for all the component-instances in the test set is provided, these labels are used to assess the performance of the AD process and create a confusion matrix. In either case, the evaluation primarily revolves around comparing the reconstructed test images against the ground truth to identify anomalies, a process that is based on comparing the results to a predefined, dynamic component-instance specific list of thresholds based on the mean reconstruction error of the validation set.

A different way to visually evaluate the training is through KDE plots that will also be generated by applying the newly trained model to unseen images. If the unseen images are all *good* examples, their error distribution should resemble that of the validation data, and if the unseen data contains anomalies, these should be positioned far from the validation distribution. In either case, both the new (orange curve) and the validation (blue curve) distribution will be displayed in a KDE plot, as seen in Figure 24.

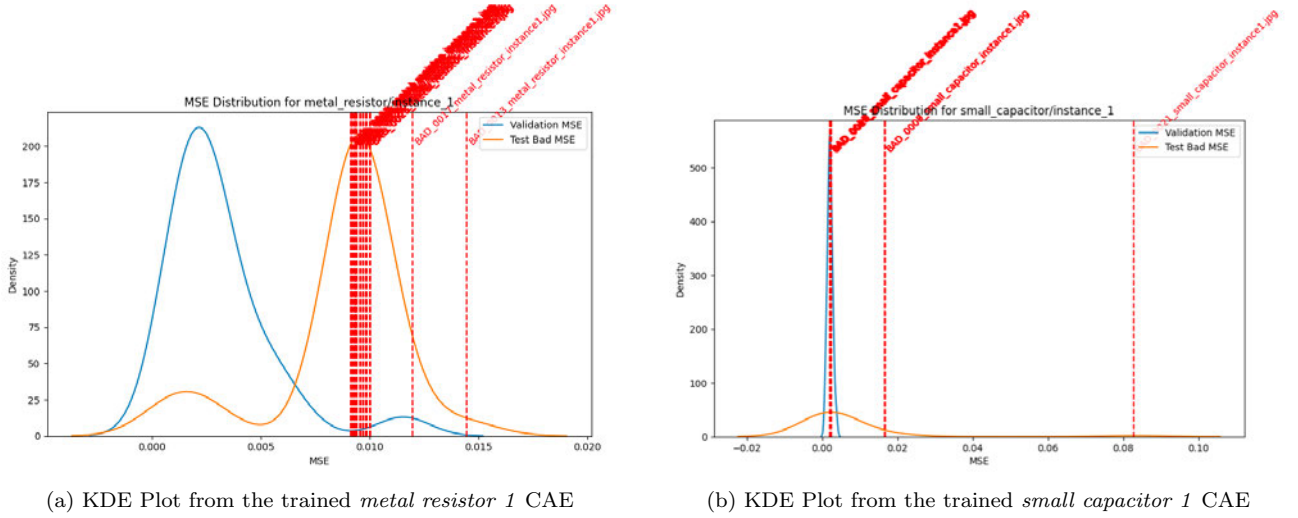


Figure 24: Two KDE Plots showing the error distributions of the validation and an anomaly-ridden dataset

Along with a vertical red line at the resulting error, the specific filename of the found anomalies is overlaid on the KDE plot, and the cropped image file along with the reconstruction loss are output to enable easy troubleshooting. Examples can be seen in Figure 25, where a correctly labelled anomalous *metal resistor 1* and an incorrectly labelled *small capacitor 1* is shown, with the latter being an example of an scenario where the training method or data should be examined.

6.2 Operational Deployment and Results Analysis

Following the setup and training, the system is ready to be applied to new PCB images by first detecting component-instances, cropping them out, and then applying the specific CAEs to the given instances in order to highlight found anomalies. A flowchart of the deployment pipeline is shown in the Appendix in Figure A2.

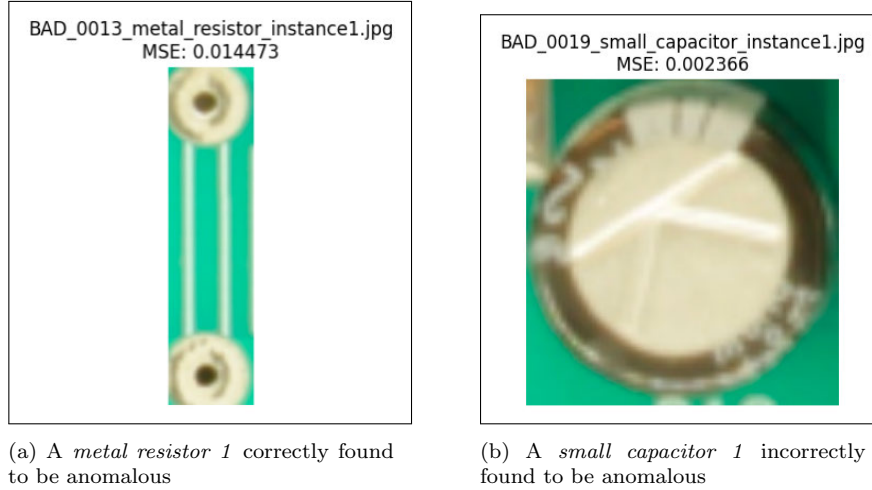


Figure 25: A correct and incorrect anomaly detection case shown after training

6.2.1 Component-Instance Identification and Model Application

The application process starts with the finetuned YOLO model looking for component-instances on the input PCB image. If one such is spotted, it is cropped out, and then the expected coordinates of component-instances, found in the baseline dictionary from Section 6.1.2, is used to determine exactly which component-instance it is by finding the instance with which it has the highest IoU, based on the coordinates of the bounding box. In case the finetuned YOLO model does not find an expected component-instance (that is, one present in the baseline dictionary), this will immediately be recorded as an anomaly, skipping the CAE application entirely, saving processing time.

After the component-instance has been determined, the specific CAE model that achieved the lowest reconstruction loss during training is located, as this indicates that it has the best performance for that particular component-instance. With the ideal model selected, the loss between the CAEs reconstruction and the newly cropped image is calculated as the resulting loss is a quantifiable measure of how closely the component matches the baseline from the validation set. If the loss is significantly higher, this indicates a potential anomaly, the gravity of which will be evaluated by pre-established thresholds.

6.2.2 Evaluating the Results

To evaluate the processing results, the pre-computed thresholds from the validation set is used to divide the anomalies into four types: Type 0 signifies no anomaly, indicating that the component is within the normal operating range as defined by the reconstruction error being below the first threshold. Types 1 through 3 represent increasing levels of anomaly severity, enabling the YOLO-CAE model to visualize the gravity of the found anomalies. A Type 1 anomaly suggests a mild deviation from the norm, captured when the reconstruction error surpasses the first threshold of mean loss plus two standard deviations but remains below the second threshold. Type 2 anomalies denote a moderate defect level, identified between the second and third thresholds, correlating to a reconstruction error between mean loss plus three and five standard deviations. Lastly, a Type 3 anomaly indicates a severe defect, flagged when the reconstruction error exceeds the mean loss plus five standard deviations. Our own testing shows that these values appropriately determine the different levels of anomalies. To reduce running times, the processing pipeline can pre-compute and save these thresholds, and only needs to do so once per dataset, or if the loss method is changed.

All detected components and their evaluation results are compiled into a comprehensive dictionary, which are saved in a uniquely timestamped directory, ensuring that each run of the pipeline is documented and traceable. This dictionary can then be compared with the true labels for each specific component-instance, if available, thus measuring the performance of the AD process. A dictionary containing the *true* labels can be generated by a function which simply adds the image path of each component-instance as the key to a dictionary, and the corresponding value is either 0 (denoting a *good* component) if the predicted anomaly is Type 0, or 1 (denoting an *anomaly*), if the predicted anomaly is either Type 1, 2, or 3. This method works best when most of the predictions are correct, but the dictionary is saved to a JSON file, such that it can be manually edited to properly label the FPs and FNs.

6.2.3 Anomaly Visualization

Upon the completion of the anomaly evaluation, each image is annotated to reflect the identified anomalies. Each bounding box is overlaid on the original PCB image according to the

coordinates specified in the final dictionary of predictions. The anomaly type determines the color of the bounding box, utilizing a simple color-coding scheme to enhance readability, enabling quick visual assessments by technicians. The colors of the bounding boxes are indicative of the anomaly severity: green for normal (Type 0), yellow for mild anomalies (Type 1), orange for moderate anomalies (Type 2), red for severe anomalies (Type 3), and grey for the instances that are so dissimilar that the YOLO cropping method discarded it.

7 Discussion

In this section we delve into the performance of the YOLO-CAE Model, starting with an assessment of its ability to accurately detect anomalies under varied conditions and instances where the model misidentified normal components or overlooked actual anomalies. Moreover, the quality of training data is critically assessed, recognizing how certain oversights can significantly impact performance. Then, the use of complex methods such as VAEs and LMMs is considered, evaluating their potential benefits against the challenges they introduce. Finally, a deep dive into the grid approach reveals why it was less successful, providing insights into methodological adjustments that could improve future implementations.

7.1 Deep Dive of Final Outputs

Figures 26b and 26a show the model's detection outputs under contrasting conditions, one riddled with multiple defects and one with a near-complete PCB, showing the model's detection granularity.

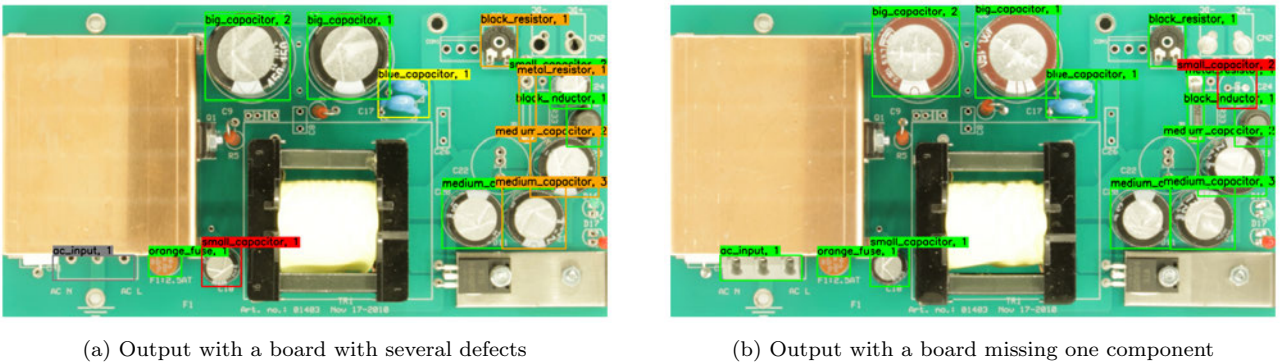


Figure 26: Two scenarios of YOLO-CAE outputs

The PCB displayed in Figure 26a contains multiple defects, resulting in a more complex scenario where, among others, *big capacitor 2* and *medium capacitor 3* are flagged, suggesting some kind of installation issue. Here, the different levels of anomaly types (see Section 6.2.3) can be seen, as the model has found anomalies of all types (0, 1, 2, and 3), corresponding to the different colours. Whether these components are incorrectly placed, improperly soldered, or entirely the wrong specification, the model brings attention to these irregularities.

On the other hand, Figure 26b is an example of the outputs from the YOLO-CAE model, where it has processed an almost complete PCB, where only a single component is missing. The model has correctly identified the absence of *ac input 1*, showing its precision in detecting missing elements on a densely populated PCB. While the model shows high performance, discerning between a defective component and the natural variation in component appearance is inherently challenging, and might fare worse in situations where the components are not so well-defined as in our scenario. To continually keep the model up to date, for example with changes to the layout, an integration into a closed-loop system where feedback from post-assembly testing can be utilized to further refine its predictive capabilities would ensure its longevity.

For instance, if a technician identifies an incorrectly classified *good* sample as a false positive, such a system would allow the model to learn to recognize this representation as a normal instance, preventing future occurrences of the same false positive. As seen in Section 7.2.2, with a CAE-based model, even a *single* image of a given *good* instance should be enough to ensure that the model avoids classifying that instance as an anomaly.

7.2 Understanding Detection Errors and Training Data Quality

AD faces challenges stemming from subtle defect patterns and the quality of training data. This section provides insights into the detection challenges and the implications of training data quality.

7.2.1 Anomaly Detection Causes

To understand the potential shortcomings and implications for ABCs quality control, a deep dive is taken into the mechanisms behind selected correct and erroneous detections. While

some errors such as missing or flipped components are immediately apparent to the naked eye, other errors are more subtle and require careful inspection to identify. This is seen in the cases of the *blue capacitors* and *black resistors* where, at first glance, these components may appear correctly installed, but a closer look reveals subtle, but critical, faults.

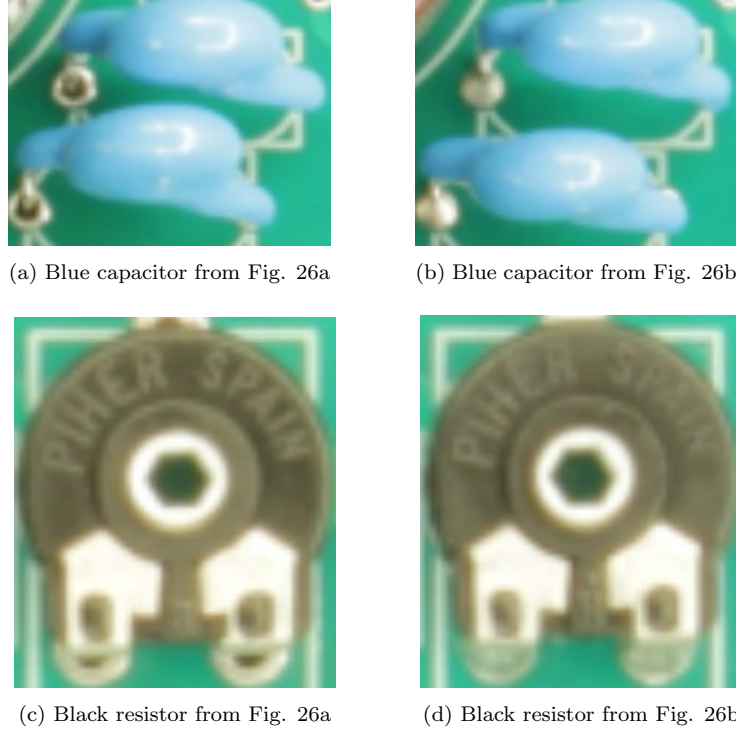


Figure 27: Components identified as defective (left) versus components appearing normal (right). Top: *Blue capacitors*; Bottom: *Black resistors*.

In Figure 27, the components from the board shown in Figure 26a (a and c) are juxtaposed with those from Figure 26b (b and d) for a side-by-side comparison. It becomes apparent that the components in Figure 26a are not properly soldered, which classifies them as anomalies.

The comparison illustrated in Figure 28 shows one of the challenges of automated PCB AD. On the left, the image shows a black inductor from the PCB that has been incorrectly classified as an anomaly. This misclassification was triggered by the absence of the adjacent *big capacitor 2*, which clips the bounding box of the black inductor to ensure that both soldering points of the inductor are included. As such, the error here lies not in the inductor itself but in the missing neighboring component that affects the spatial context within which the inductor is evaluated.

On the right, the same inductor is correctly identified as non-anomalous, despite the fact that a

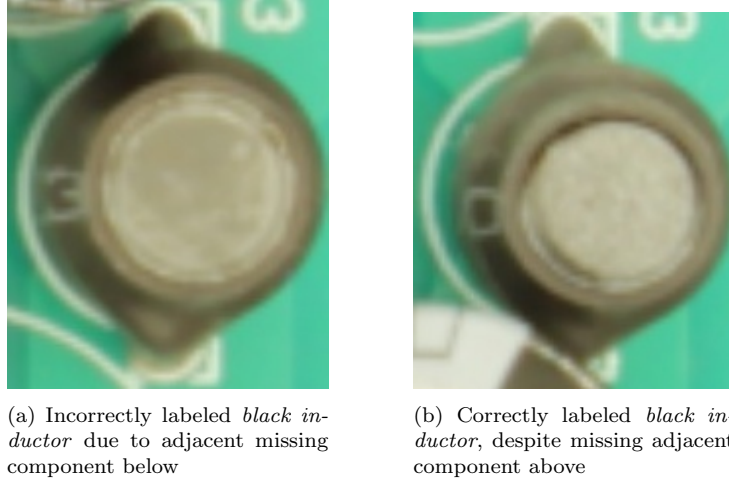


Figure 28: Comparison of black inductor detection with an adjacent component anomaly (left) versus accurate detection (right)

small capacitor, specifically instance 2, is missing above. In cases like this where the bounding box of one component necessarily clips that of another, false positives are likely to occur. In a real life scenario, this does not have a significant effect on the outcome, as it simply further highlights the fact that *some* anomaly is present in that area.

7.2.2 The Effects of Imperfect Training Data

During the initial training, it was observed that the *small capacitor 2 CAE* failed to correctly identify missing components as anomalies, due to the fact that a PCB (the one depicted in Figure 26b), supposed to be marked as defective due to a missing component, was erroneously included in the training dataset as a non-defective example.

Despite the training dataset comprising 200 highly homogeneous examples for *small capacitor 2*, the inclusion of just one anomalous image led the CAE to learn that both the standard and the defective states of the component were normal. This type of learning anomaly underscores the sensitivity of this CAE architecture to its training data, especially as when training with a small dataset, the model may have sufficient capacity to memorize and reproduce the specific features of a single anomalous representation.

To address this issue, the CAE was retrained without the erroneous example, and a comparative analysis of the model's performance, pre and post-correction, was conducted using both 'good' (non-defective) and 'bad' (defective) examples of the capacitor.



Figure 29: Reconstructions of a missing *small capacitor 2* by CAEs with and without the spurious training example, illustrating the importance of correct training data

For 'good' examples, where the capacitor was present and correctly installed, the MSE loss remained largely consistent before and after the correction of the training data 0.0025 vs. 0.0027. This consistency indicates that the model's ability to recognize non-defective components was not compromised by the exclusion of the spurious example. However, the scenario was different when evaluating the example where the capacitor was missing. Prior to retraining, the MSE loss for a missing capacitor was only slightly higher than the 'good' example, at 0.0035, still being a failure in detecting the anomaly. However, post-retraining, the MSE loss increased significantly to 0.0643, a near twenty-fold rise, clearly marking it as an anomaly.

This change in MSE loss values post-retraining underlines the importance of precise and accurate training datasets in AD systems. For ABC, this finding emphasizes the need for meticulous data handling and preprocessing to ensure the reliability of this type of system.

7.3 Implications of Utilizing YOLO

Every extra step of a process has a cost, and as such, the integration of the YOLO architecture in the YOLO-CAE Model must be justified, as opposed to using a fixed template to crop out components.

7.3.1 Advantages of Integrating YOLO with CAEs

The results clearly show that using the YOLO model to identify ROIs leads to more reliable data for training the CAEs, as well as better cropping when applying those same CAEs to new images. YOLO's detection algorithms are particularly good at accurately marking the edges of components, even when their shapes are irregular due to manual hand placement, which is regularly the case with our dataset. This precision is also needed when dealing with components like tall capacitors or metal resistors, which due to natural variance, may not always seem to be perfectly aligned. Furthermore, the consistency between the data prepared by YOLO and what the CAE is applied on matters. When the CAE receives data that is uniform and correctly aligned, it can predict more effectively as it doesn't have to struggle with correcting errors from skewed or incorrectly cut-out components, making them better at telling the difference between normal component variations and actual defects.

Additionally, by using YOLO before the CAEs step in, the system gains an extra layer of reliability as YOLO evaluates how confident it is in its detections and only crops out and passes on the components that somewhat resemble the original ones for the CAE to examine further. As such, this approach immediately labels the low-confidence components as anomalies, bypassing the given CAE and thus saving time. This almost exclusively happens when a component is missing.

7.3.2 Comparative Performance Analysis

The assessment of the YOLO-TC and YOLO-CAE models, detailed in Table 6, shows distinct differences in performance metrics. A stark contrast is evident in the precision and specificity measures between the two models. The YOLO-CAE model achieves an F1-Score of 0.99 in contrast to the TC-CAE model at a significantly lower 0.91. This discrepancy is primarily

attributed to the TC-CAE model’s higher false positive rate, which not only increases the likelihood of unnecessary rework but also introduces potential delays in the manufacturing process due to unwarranted inspections.

For a closer per-component look, see Appendix Table A2. When looking at the component-specific data, it is evident that the Medium Capacitor component has notably influenced the overall performance discrepancy between the two models. In the TC-CAE system, this component alone accounted for a substantial proportion of the false positives. This high rate of false detections suggests a potential misalignment or inadequate training of the TC model specifically for this component type.

Moreover, the black inductor is another interesting case. The YOLO-CAE model finds half the amount of false positives compared to the TC-CAE model, which demonstrates a precision of only 0.20 for this component, further emphasizing the TC model’s higher tendency to incorrectly flag non-defective parts as defective, especially in components with higher variability in appearance, and in this case, variability in the surrounding components too (see Figure 28a).

7.4 Implications of Using Complex Approaches

While the applied methods in this paper perform very well, this is all to be seen in the context of the controlled environment and relatively easily distinguishable anomalies. Should these types of anomalies continue to be the focus for this part of the quality control for ABC, the proposed method need only a retraining with images of new PCB designs. However, should the scope be widened to cover less clean-cut anomalies, more complex methods should be considered, such as VAEs and LMMs.

7.4.1 Using VAEs for AD

Even though the CAEs provided clear differentiation between normal and faulty instances in our controlled testing environment, where we had a representative set of around 200 training images per component, a potential limitation of this approach is the generalization under more complex scenarios, or scenarios with fewer training samples. For instance, consider the example of detecting slight positional shifts in text printed on components. While a CAE trained on

specific text alignments (like centered or slightly offset to the left or right) can reliably detect those cases, it might struggle with unseen alignments due to its direct reconstruction approach. Thus, if the component text is misaligned by an amount not explicitly seen during training, the CAE might misclassify this as anomalous.

In contrast, the earlier discussed VAE could offer an advantage here as the VAEs probabilistic framework allows it to model the continuous variation of "good" samples. By learning the overall statistical distribution of normal patterns, the VAE can interpolate between observed variations and better identify unseen but similar samples as normal. In this case, slight text misalignment might fall within the statistical distribution, allowing the VAE to classify them correctly. The VAEs latent space makes it particularly advantageous in more complex scenarios where slight positional or stylistic deviations can occur, or when fewer training samples are available to capture the full range of "good" patterns. As such, **ABC** could use the generalization capabilities of VAEs to produce even more robust solutions for AD.

7.4.2 Using LMMs for AD

Similarly to VAEs, while LMMs like GPT-4V show promise for AD, several factors make them beyond the scope of this paper. First, these models are very computationally intensive, leading to higher training and inference costs. Additionally, training time is significant due to their expansive architectures and data requirements, which can make it difficult to achieve real-time performance or scale the system efficiently.

Furthermore, LMMs are inherently complex, often behaving as black boxes that are hard to interpret and analyze. This lack of interpretability complicates debugging and tuning, potentially impacting operational stability and delaying troubleshooting in a production environment. For example, it can be challenging to discern why a particular anomaly was missed or why a false positive was flagged. Such diagnostic difficulty can hinder rapid response in quality control. Also, due to their intricate structure, LMMs can overfit when training on a small dataset. Their sophisticated architectures are prone to learning patterns specific to training data, reducing their ability to generalize to new samples. This overfitting problem is exacerbated when the data is scarce, which makes it difficult to establish a representative training set that can

cover all potential anomalies.

Despite these challenges, ABC could consider exploring LMMs in the future, as these models could complement other approaches and facilitate a more thorough multi-modal AD approach. Their advanced capabilities in processing visual as well as other forms of data could enable deeper insights into the likely complex relationships between component defects and manufacturing patterns. Further, if also combined with more interpretable models, this could provide a comprehensive AD system that leverages the strengths of each model type.

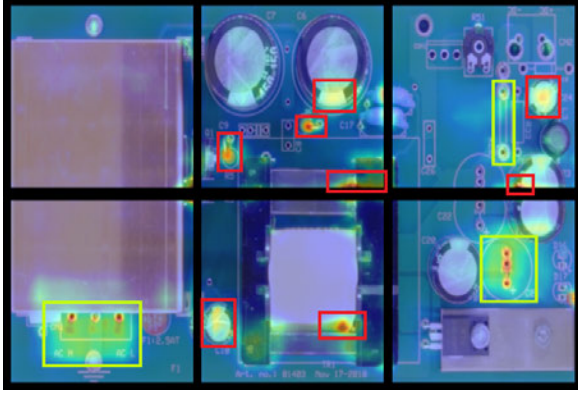
7.5 Implications of Grid-Wise and Perceptual Methods

While the grid-wise approach produced among the worst performing models, examining the results closer do provide potential valuable insights for ABC. A visual analysis of what the grid models emphasize aids in understanding the roots of the relatively poor performances.

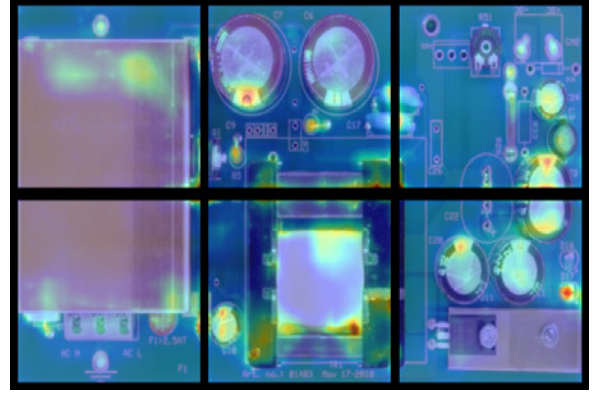
7.5.1 Main Model

The heatmaps displayed in Figure 30 are created from a PCB with anomalies on the left and a PCB without any errors on the right in order to better understand what catches the model’s attention. The heatmaps are composites of individual heatmaps created by calculating the differences between the input image and the reconstructed image for each selected layer used to extract features from VGG19. A brighter and stronger color closer to red indicates a stronger deviation from the expected content. In Figure 30, actual anomalies are highlighted with green boxes and red boxes are drawn around areas where nothing is wrong. It is evident that the model has failed to generalize normal features, such as the surfaces of components, and the inherent variance in positioning/angle of smaller items, such as in the upper middle grid of Figure 30a.

Even when using later layers of VGG19 to emphasize larger features, and a heavy weighting of the perceptual component, 95%, the model still gives natural variance regarding placement of components and small texture differences high importance. This was a recurring issue for all layer configurations that were experimented with. Thus, when computing the reconstruction error for any given grid, the total amount of error that may be present contributes to difficulties



(a) Highlighted differences on PCB with anomalies



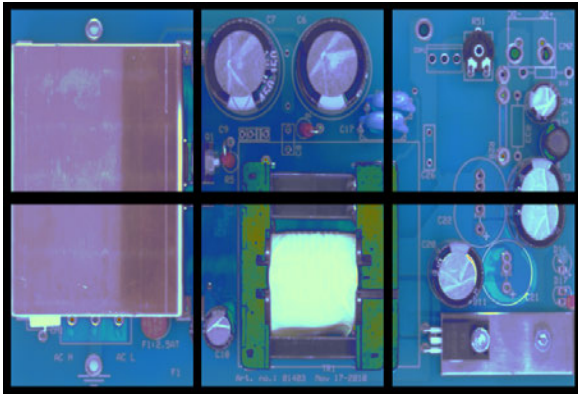
(b) Highlighted differences on PCB without anomalies

Figure 30: Visualization of bad versus normal PCB with a perceptual model using later layers

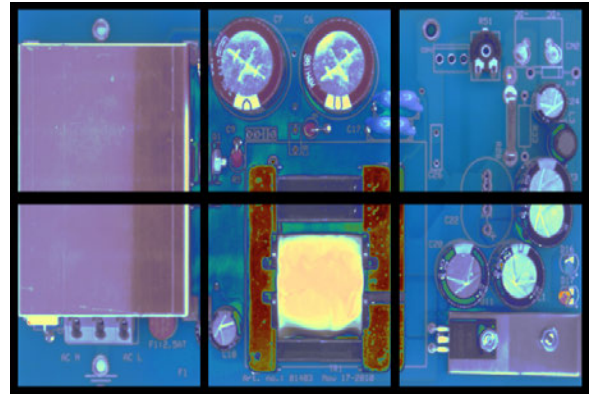
in differentiating between anomaly-free and anomaly-present images.

7.5.2 Baseline Model

Concerning the base line model using solely pixel-wise differences, while the precision and recall scores are close enough to compete with the perceptual model, a visual analysis reveals similar issues concerning the natural variances. Figure 31 shows the heatmaps for the same input images as in Figure 30. In the upper middle grid, for both images the components are positioned correctly, but, for instance, in Figure 31b, the model highlights minor differences in texture on the good PCB, whereas it does not on the bad PCB. Next, while a component is missing in both the lower left and right grid in Figure 31a, the heatmaps indicates that these differences are smaller than, for example, the differences highlighted in the lower right grid in Figure 31b, which are due to negligible surface variations on top of the conductors.



(a) Highlighted differences on PCB with anomalies



(b) Highlighted differences on PCB without anomalies

Figure 31: Visualization of bad versus normal PCB without the perceptual component

7.5.3 Comparison

Both heatmap comparisons indicates several things. First, the size of the grids selected is too large, enabling the natural variances inherent in the good PCBs to interfere with actual anomalies, effectively creating blind-spots. Second, the selection of perceptual layers for feature extraction were inadequate, in terms of picking up more subtle differences, such as the small directional marker on the black conductor in the middle lower grid, or the missing metal pin in the upper right grid, while at the same time ignoring minor, natural variances of similar sizes such as the yellow tape in the middle grid (see Figure 18e for an example). This also reflects sub-optimal representations in the latent spaces of these models, as to encapsulate the normal distribution of the PCB distinctively from actual outliers. Finally, an interesting thing to note when examining the heatmaps is the disregard for maintaining spatial dimensions when cropping and resizing grids appears to be of less importance.

Model	Threshold	Precision	Recall	F1
GM-CAE	1	.84	.81	.82
	2	.80	.61	.69
	3	1	.30	.46
	4	1	.11	.20
GP-CAE	1	.60	1	.75
	2	.68	.98	.80
	3	.69	.94	.80
	4	.88	.81	.84

Table 7: Performance at different threshold levels

Next, both models performance metrics suggest that adapting the configuration for easier integration into operational environments and accommodating future product lines results in a trade-off between precision and recall. Table 7 shows the respective performances at the different threshold levels. For GM-CAE, there is a decline in F1-score as the number of standard deviations increases, whereas it is the opposite pattern for the GP-CAE. This suggests that the perceptual component help accentuate differences from the good distribution (and create more meaningful visualizations), in contrast to a more concentrated distribution when considering purely pixel-differences. Nevertheless, the F1-scores indicate that opting for more manageable grid sizes (larger) to reduce manual data processing and labeling leads to models where one

achieves high precision or high recall, but not both simultaneously.

de Oliveira et al. (2023) findings suggest a potential for a system that does not rely on narrowly confined ROIs, thus allowing undefined anomalies to be detected, although this would require further experiments with grid sizes and VGG19 layer configurations to confirm applicability when anomalies may have such big differences in size. However, considerably smaller grids significantly increase the manual workload, which contrasts one of the desired features of using deep learning for this project, avoiding extensive adaptations to each PCB (Chalapathy & Chawla, 2019). Given the extensive manual processing required for this relatively small-scale PCB project compared to the more successful YOLO approach, respective annotation time and overall performance considered, further exploration of global AD approaches may not be of interest for ABC depending on the likelihood of none-hand-mounted anomalies.

8 Limitations

As the scope of this project is narrowed down to anomalies within only hand-mounted components within the context of ABC, there are several limitations to the applicability of our approach.

Transferability and Speed Although our work produces promising results, its transferability to a production environment and other PCB products at ABC remains unproven. The current system is tailored to, and tested under, a specific scenario and subset of known anomaly types. Additionally, the system was tested mostly using an unsoldered PCB to replicate failures. This does not fully represent real-world conditions of PCBs with naturally occurring anomalies, including properly soldered components, which could yield different performance and insights. Further testing of various PCB types, covering a greater range of components, sizes, and error scenarios is necessary for a thorough understanding of the system’s applicability. This requires time, material, and financial investments, adding to the cost-benefit analysis for the system as a whole. All of this demands a certain level of domain- and product-specific knowledge, which contributes to constraints for further assessment and development of the system. Furthermore, the run time for processing PCBs depends on the specific hardware setup and software im-

plementation at ABC, making it hard to determine if the system will be fast enough for a production environment.

Camera Setup Concerning the hardware aspect of the setup, there are multiple potentially limiting aspects. The top-down camera angle, especially with smaller focal lengths, can result in important parts of components being obscured by others. For instance, a tall capacitor can clip the soldering point for a nearby resistor (as is seen in Section 7.2.1). Increasing the focal length of the lens can mitigate this issue and improve detection accuracy, as elaborated on in Section 4.4.1. Future system designs should include adjustable camera setups to ensure that all critical components are clearly visible, reducing the likelihood of missed or incorrect detections. As such, factors such as the placement and type of lighting and the choice of lens must be considered for future system development. The foundation laid by our current setup is promising, but further evaluation is needed to determine its adaptability to different PCB configurations and sizes, particularly as the current dimension of the boxes does not accommodate considerably larger PCBs.

Further Model Exploration The field of AD is a vast and active research domain, therefore producing a comprehensive review covering all relevant models and methods of interest was infeasible. Due to limited time and resources, and on the background of strong test results, our explorations did not extend beyond the simpler model architectures CNN and CAE. While more complex alternatives, which may provide versatility for more complex PCB cases, are discussed in Section 7.4, our study finds that the simpler models can be capable of performing well enough for the specific needs of ABC. However, considering that the current system requires several individual models to be trained for each ROI on any given PCB, and that the number ROIs reasonably may become dozens, investigating both alternative approaches as well as model architectures in terms of efficiency may be relevant. This could yield both faster training and more robust models. As such, future work could consider the alternative one-class models outlined in 3.2.2, in addition to a broader range of AD methods.

9 Conclusion

This thesis has successfully demonstrated the viability of an ensemble computer vision approach for detecting anomalies in hand-mounted components on PCBs. In collaboration with the partner company ABC, we developed a targeted solution that serves as a practical and cost-effective alternative to the expensive, off-the-shelf AOI systems. These traditional systems often fall short in addressing the specific challenges associated with the manual component mounting phase of PCB assembly.

Addressing the research question,

How can a cost-efficient hardware and software setup be designed and implemented for anomaly detection in hand-mounted PCB components for small-manufacturers, considering the specific needs and constraints given by ABC?

this work presents a POC that ABC can continue to refine and expand in-house for a fraction of the cost (less than half of the budget) compared to industrial AOI alternatives. Our approach integrated a YOLO Object Detection model with a system that trains specialized Convolutional Autoencoders tailored to each individual component of interest. This method proved effective in recognizing subtle anomalies that can be missed during manual inspections. While this POC provides strong results, several areas require further exploration, experimentation, and testing to ensure broad applicability and consistent performance. Still, the upgraded camera system, specialized models, and surrounding training and operational pipelines integrate to form the POC for a robust system, giving ABC a framework to further develop this technology, optimizing it to better suit their specific production environments and possibly extending its application to other types of PCBs.

References

- Abellan-Nebot, J. V., & Subirón, F. R. (2010). A review of machining monitoring systems based on artificial intelligence process models. *The International Journal of Advanced Manufacturing Technology*, 47(1), 237–257.
- Aggarwal, C. C. (2023). *Neural networks and deep learning: A textbook, 2nd edition* (2nd ed.). Springer International Publishing.
- An, J., & Cho, S. (2015). Variational autoencoder based anomaly detection using reconstruction probability. <https://api.semanticscholar.org/CorpusID:36663713>
- Beak, S., Han, Y.-H., Moon, Y., Lee, J., & Jeong, J. (2023). Yolov7-based anomaly detection using intensity and ng types in labeling in cosmetic manufacturing processes. *Processes*, 11(8). <https://doi.org/10.3390/pr11082266>
- Bergmann, P., Löwe, S., Fauser, M., Sattlegger, D., & Steger, C. (2019). Improving unsupervised defect segmentation by applying structural similarity to autoencoders, 372–380.
- Cao, Y., Xu, X., Sun, C., Huang, X., & Shen, W. (2024). Towards generic anomaly detection and understanding: Large-scale visual-linguistic model (gpt-4v) takes the lead. *arXiv preprint arXiv:2401.12345*.
- Chalapathy, R., & Chawla, S. (2019). Deep learning for anomaly detection: A survey. *arXiv preprint arXiv:1901.03407*.
- Chalapathy, R., Menon, A. K., & Chawla, S. (2019). Anomaly detection using one-class neural networks. *arXiv preprint arXiv:1902.04359*.
- Chandola, V., Banerjee, A., & Kumar, V. (2007). Outlier detection: A survey. *ACM Computing Surveys*, 14, 15.
- Chapman, P. (2000). Crisp-dm 1.0: Step-by-step data mining guide. <https://api.semanticscholar.org/CorpusID:59777418>
- Chen, T., Liu, X., Xia, B., Wang, W., & Lai, Y. (2020). Unsupervised anomaly detection of industrial robots using sliding-window convolutional variational autoencoder. *IEEE Access*, 8, 47072–47081. <https://doi.org/10.1109/ACCESS.2020.2979310>
- De Albuquerque Filho, J. E., Brandão, L. C., Fernandes, B. J. T., & Maciel, A. M. (2022). A review of neural networks for anomaly detection. *IEEE Access*, 10, 112342–112367.

- de Oliveira, D. C., Nassu, B. T., & Wehrmeister, M. A. (2023). Image-based detection of modifications in assembled pcbs with deep convolutional autoencoders. *Sensors*, *23*(3), 1353. <https://doi.org/10.3390/s23031353>
- Fischler, M. A., & Bolles, R. C. (1981). Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, *24*(6), 381–395.
- Gatys, L. A., Ecker, A. S., & Bethge, M. (2016). Image style transfer using convolutional neural networks. *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2414–2423.
- Géron, A. (2023). *Hands-on machine learning with scikit-learn, keras, and tensorflow concepts, tools, and techniques to build intelligent systems* (3rd ed.). O'Reilly Media, Inc. libgen.li/file.php?md5=2391108a195cb4b1dd5c61c785d376a2
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 249–256.
- Google Brain Team. (2024). Convolutional autoencoders.
- Goto, K., Kato, K., Saito, T., & Aizawa, H. (2020). Adversarial autoencoder for detecting anomalies in soldered joints on printed circuit boards. *Journal of Electronic Imaging*, *29*(4), 041013. <https://doi.org/10.1117/1.JEI.29.4.041013>
- Hamdi, S., Ghanem, B., & Cao, L. S. (2020). End-to-end deep one-class learning for anomaly detection in uav video stream. *IEEE Transactions on Image Processing*, *29*, 6664–6677.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. *Proceedings of the IEEE international conference on computer vision*, 1026–1034.
- Iman, M., Rasheed, K., & Arabnia, H. R. (2023). A review of deep transfer learning and recent advancements.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International conference on machine learning*, 448–456.

- Kim, J., Ko, J., Choi, H., & Kim, H. (2021). Printed circuit board defect detection using deep learning via a skip-connected convolutional autoencoder. *Sensors (Basel)*, 21(15), 4968. <https://doi.org/10.3390/s21154968>
- Kingma, D. P., & Ba, J. (2017). Adam: A method for stochastic optimization.
- Krizhevsky, A. (2009). *Learning multiple layers of features from tiny images* (tech. rep.). University of Toronto. <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 1097–1105.
- LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4), 541–551.
- LeCun, Y., Cortes, C., & Burges, C. J. (1998). The mnist database of handwritten digits [Accessed: 02/03/2024].
- Lin, T.-Y., Patterson, G., Ronchi, M. R., Cui, Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, L., & Dollár, P. (2021). Coco: Common objects in context dataset [Accessed: date-of-access]. *COCO Consortium*.
- Liu, H., & Wang, H. (2023). Real-time anomaly detection of network traffic based on cnn. *Symmetry*, 15(6). <https://doi.org/10.3390/sym15061205>
- Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. *International journal of computer vision*, 60, 91–110.
- Makhzani, A., Shlens, J., Jaitly, N., Goodfellow, I., & Frey, B. (2016). Adversarial autoencoders.
- Mirapeix, J., et al. (2007). Automated optical inspection system for bare pcbs. *Electronics Letters*, 43(8), 956–958.
- OpenAI. (2023). GPT-4V(ision) System Card [Version 1].
- OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., . . . Zoph, B. (2024). Gpt-4 technical report.

- Ruff, L., Kauffmann, J. R., Vandermeulen, R. A., Montavon, G., Samek, W., Kloft, M., Dietterich, T. G., & Müller, K.-R. (2021). A unifying review of deep and shallow anomaly detection. *Proceedings of the IEEE*, 109(5), 756–795.
- Ruff, L., Vandermeulen, R. A., Görnitz, N., Deecke, L., Siddiqui, S. A., Binder, A., Müller, E., & Kloft, M. (2018). Deep one-class classification. *Proceedings of the 35th International Conference on Machine Learning*, 80.
- Saunders, M. N. K., Lewis, P., & Thornhill, A. (2023). *Research methods for business students* (9th ed.).
- Schneider, S., Antensteiner, D., Soukup, D., & Scheutz, M. (2022). Autoencoders - a comparative analysis in the realm of anomaly detection, 1985–1991. <https://doi.org/10.1109/CVPRW56347.2022.00216>
- Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.
- Son, V. H., Daisuke, U., Kiyoshi, H., Kazuki, M., Pranata, S., & Shen, S. M. (2020). Anomaly detection with adversarial dual autoencoders. *Proceedings of the Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Ultralytics. (2024). Ultralytics yolov8 documentation.
- Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). Image quality assessment: From error visibility to structural similarity. *IEEE transactions on image processing*, 13(4), 600–612.
- Waskom, M., & the Seaborn development team. (n.d.). Seaborn.kdeplot [Accessed: 2024-05-01].
- Xu, B., Wang, N., Chen, T., & Li, M. (2015). Empirical evaluation of rectified activations in convolutional network. *arXiv preprint arXiv:1505.00853*.
- Zhang, J., He, H., Chen, X., Xue, Z., Wang, Y., Wang, C., Xie, L., & Liu, Y. (2023). Gpt-4v-ad: Exploring grounding potential of vqa-oriented gpt-4v for zero-shot anomaly detection. *arXiv preprint arXiv:2402.98765*.

A Appendix

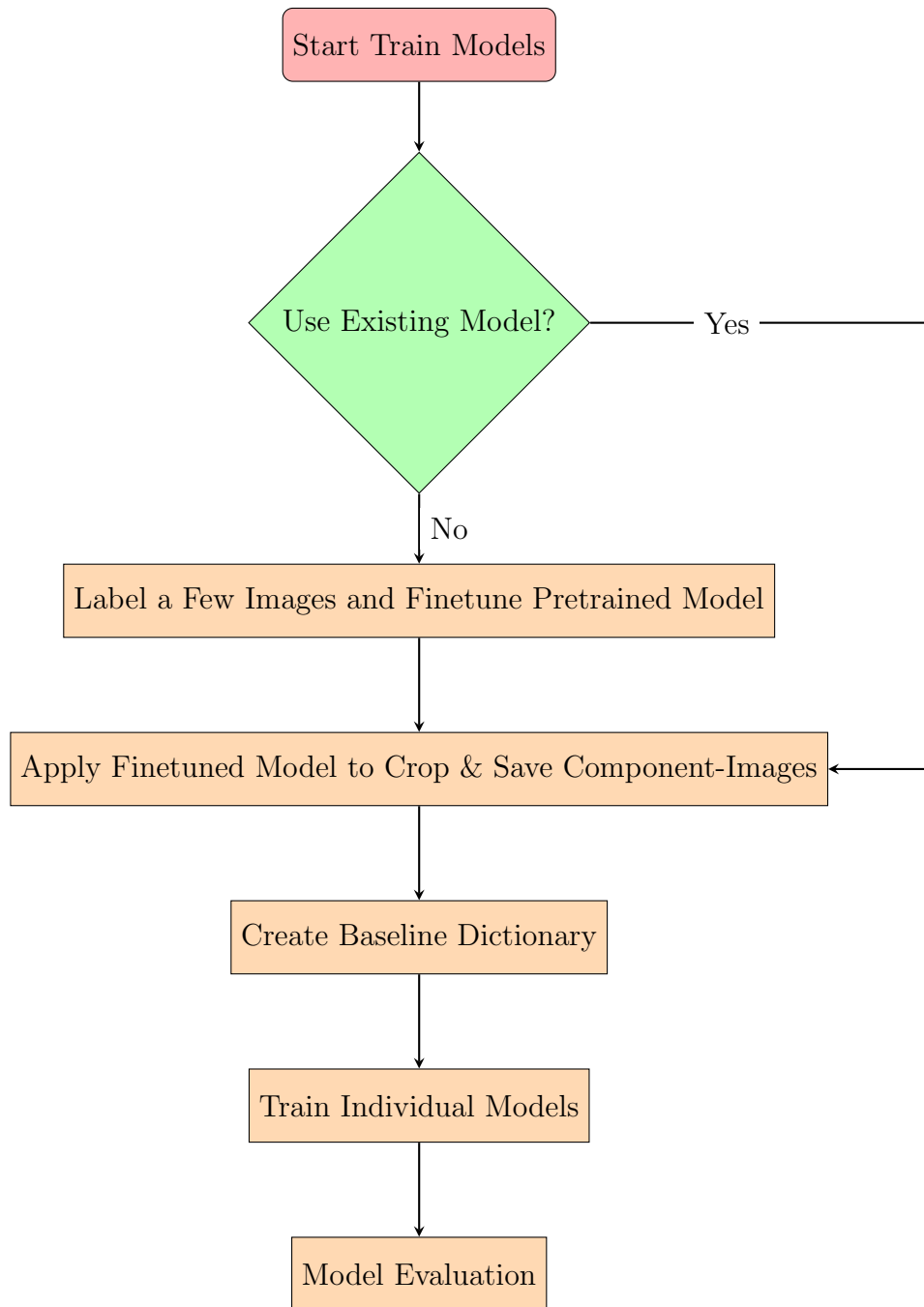


Figure A1: Training Pipeline Flowchart

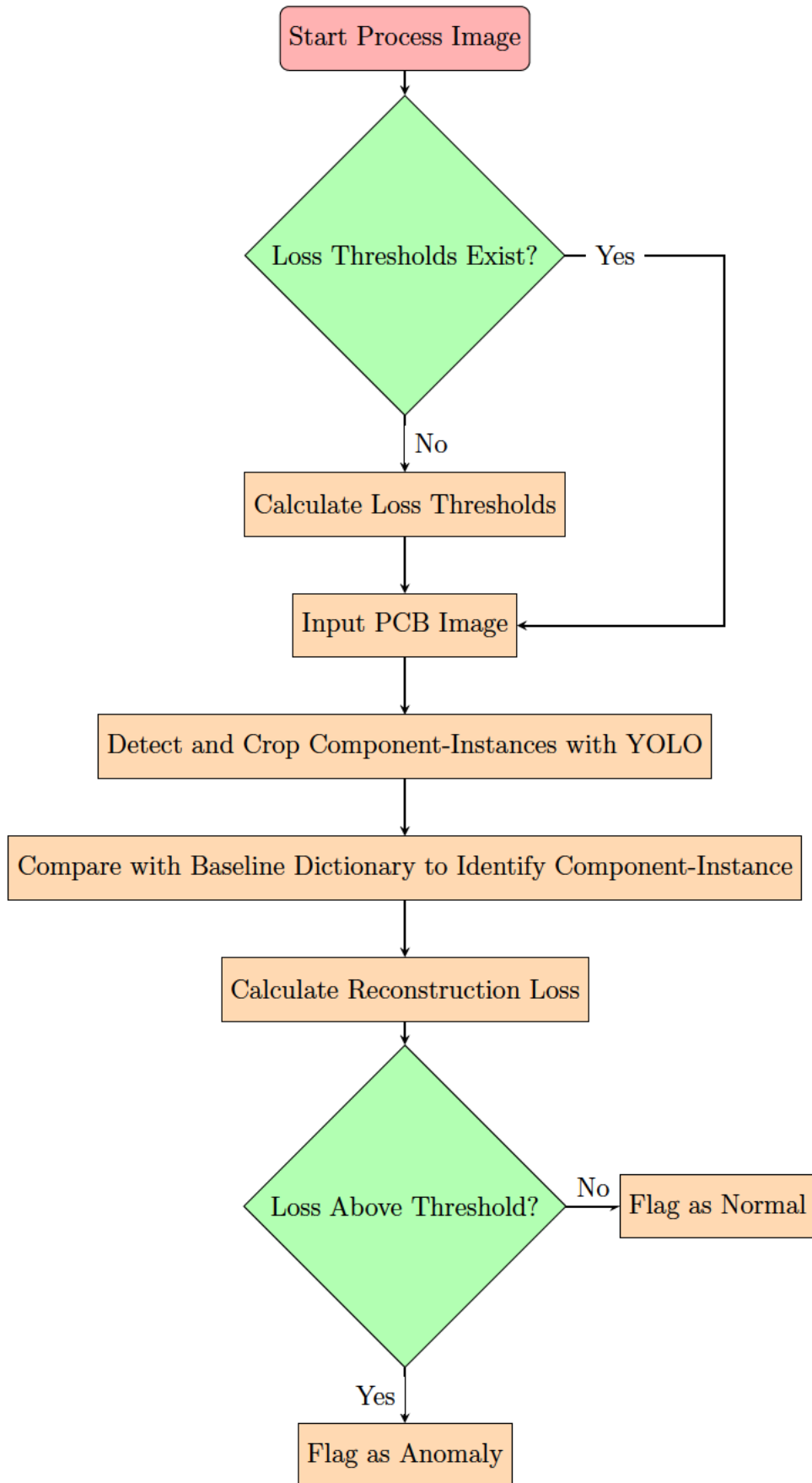


Figure A2: Deployment Pipeline Flowchart

Table A1: Performance metrics for each model configuration and threshold value (Number of standard deviations away from mean)

Layers	Perceptual Weight	Threshold	Accuracy	Precision	Recall	F1
None	.00	1	0.84	0.84	0.81	0.82
		2	0.75	0.80	0.61	0.69
		3	0.68	1.00	0.30	0.46
		4	0.59	1.00	0.11	0.20
Early	.95	1	0.69	0.60	1.00	0.75
		2	0.72	0.62	1.00	0.77
		3	0.80	0.74	0.87	0.80
		4	0.79	0.89	0.61	0.73
Early	.99	1	0.68	0.59	1.00	0.74
		2	0.71	0.61	1.00	0.76
		3	0.73	0.66	0.88	0.75
		4	0.82	0.92	0.67	0.78
Middle	.95	1	0.68	0.59	1.00	0.74
		2	0.72	0.62	1.00	0.77
		3	0.71	0.64	0.87	0.73
		4	0.82	0.95	0.65	0.77
Middle	.99	1	0.66	0.58	1.00	0.73
		2	0.72	0.62	1.00	0.76
		3	0.78	0.70	0.94	0.80
		4	0.84	0.88	0.76	0.81
Late	.95	1	0.69	0.60	1.00	0.75
		2	0.77	0.68	0.98	0.80
		3	0.78	0.69	0.94	0.80
		4	0.86	0.88	0.81	0.84
Late	.99	1	0.66	0.58	1.00	0.73
		2	0.73	0.63	1.00	0.77
		3	0.73	0.65	0.90	0.75
		4	0.76	0.79	0.65	0.72
All	.95	1	0.65	0.57	1.00	0.72
		2	0.75	0.66	0.94	0.78
		3	0.68	0.63	0.72	0.67
		4	0.70	0.72	0.57	0.64
All	.99	1	0.68	0.59	1.00	0.74
		2	0.72	0.62	1.00	0.76
		3	0.79	0.77	0.78	0.78
		4	0.80	0.93	0.61	0.74

Table A2: Per-component performance metrics across models

Component	Metric	YOLO-CNN	TC-CAE	YOLO-CAE
Medium Capacitor	Precision	0.60	0.45	1.00
	Recall	0.88	1.00	1.00
	Specificity	0.86	0.66	1.00
	Accuracy	0.86	0.73	1.00
Orange Fuse	Precision	1.00	1.00	1.00
	Recall	1.00	1.00	1.00
	Specificity	1.00	1.00	1.00
	Accuracy	1.00	1.00	1.00
AC Input	Precision	1.00	1.00	1.00
	Recall	1.00	1.00	1.00
	Specificity	1.00	1.00	1.00
	Accuracy	1.00	1.00	1.00
Blue Capacitor	Precision	1.00	1.00	1.00
	Recall	1.00	1.00	1.00
	Specificity	1.00	1.00	1.00
	Accuracy	1.00	1.00	1.00
Black Resistor	Precision	1.00	1.00	1.00
	Recall	0.26	1.00	1.00
	Specificity	1.00	1.00	1.00
	Accuracy	0.35	1.00	1.00
Big Capacitor	Precision	1.00	1.00	1.00
	Recall	1.00	1.00	1.00
	Specificity	1.00	1.00	1.00
	Accuracy	1.00	1.00	1.00
Black Inductor	Precision	0.20	0.20	0.33
	Recall	1.00	1.00	1.00
	Specificity	0.86	0.86	0.93
	Accuracy	0.87	0.87	0.93
Small Capacitor	Precision	0.58	0.88	0.89
	Recall	0.88	1.00	1.00
	Specificity	0.90	0.98	0.98
	Accuracy	0.90	0.98	0.98
Metal Resistor	Precision	1.00	1.00	1.00
	Recall	1.00	1.00	1.00
	Specificity	1.00	1.00	1.00
	Accuracy	1.00	1.00	1.00